

# Tecniche di programmazione Funzionale ed Imperativa

Appunti del Corso

Alessia Cassetta

31 marzo 2026

# Indice

---

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>La Babele dei Linguaggi di Programmazione</b>                      | <b>1</b>  |
| 1.1      | Dagli Algoritmi alla Pratica: Cosa può andare storto? . . . . .       | 1         |
| 1.2      | Computabilità e Astrazione . . . . .                                  | 1         |
| 1.3      | L'Evoluzione dei Paradigmi di Programmazione . . . . .                | 2         |
| 1.4      | Programmazione Funzionale e Haskell . . . . .                         | 2         |
| 1.4.1    | Tipi, Costruttori e Notazione Infissa . . . . .                       | 3         |
| 1.4.2    | Ordine Superiore e Composizionalità . . . . .                         | 4         |
| 1.4.3    | Il calcolo dei programmi: Valutazione Ricorsiva e undefined . . . . . | 4         |
| <b>2</b> | <b>Quegli Strani Tipi (Polimorfi) e Funzioni</b>                      | <b>5</b>  |
| 2.1      | Type Inference e la Funzione Identità . . . . .                       | 5         |
| 2.2      | Termini non Tipabili, Non-Terminazione e undefined . . . . .          | 6         |
| 2.3      | Costrutti, Pattern Matching e Tuple . . . . .                         | 6         |
| 2.3.1    | L'evoluzione del NOT e il parametro anonimo . . . . .                 | 7         |
| 2.3.2    | Funzioni Binarie e il Divieto di Unificazione . . . . .               | 7         |
| 2.3.3    | Pattern Matching sui Numeri Naturali . . . . .                        | 7         |
| 2.3.4    | Decomposizione di Tuple e Lambda . . . . .                            | 8         |
| 2.4      | Programmazione su Liste . . . . .                                     | 9         |
| 2.4.1    | Pattern Matching: Testa, Coda e Parentesi . . . . .                   | 9         |
| 2.4.2    | Ispezione ed Esplorazione: null, last e init . . . . .                | 9         |
| 2.4.3    | L'operatore di Concatenazione (++) . . . . .                          | 10        |
| 2.4.4    | Estrazione e Scomposizione (take, drop, splitAt, !!) . . . . .        | 10        |
| 2.4.5    | Liste Annidate: zip e concat . . . . .                                | 11        |
| <b>3</b> | <b>Verso un Ordine Superiore: Funzionali e Applicazioni</b>           | <b>12</b> |
| 3.1      | Program Calculation: Valutazione passo-passo . . . . .                | 12        |
| 3.2      | Zucchero Sintattico: Le Enumerazioni . . . . .                        | 12        |
| 3.3      | Curryficazione e Manipolazione degli Argomenti . . . . .              | 13        |
| 3.4      | Schemi di Programmazione: map e filter . . . . .                      | 13        |
| 3.5      | List Comprehension: La Matematica in Haskell . . . . .                | 14        |
| 3.6      | Il Prodotto Scalare: Un Trionfo dell'Ordine Superiore . . . . .       | 14        |
| 3.7      | Esistenziale ed Universale: any, all e la Ricerca su Liste . . . . .  | 15        |
| 3.8      | I Funzionali di Riduzione: foldr e foldr1 . . . . .                   | 15        |
| 3.9      | Applicazioni Avanzate: Matrici e Verifica di Ordinamento . . . . .    | 16        |
| <b>4</b> | <b>Tipi Strani, ma di Classe</b>                                      | <b>17</b> |
| 4.1      | Classi e Subtyping: Il Polimorfismo Vincolato . . . . .               | 17        |
| 4.2      | La Gerarchia delle Classi Predefinite . . . . .                       | 18        |
| 4.2.1    | La Classe Eq (Uguaglianza) . . . . .                                  | 18        |
| 4.2.2    | La Classe Ord (Ordinamento) . . . . .                                 | 18        |
| 4.2.3    | Le Classi Num, Show e Read . . . . .                                  | 18        |
| 4.3      | Costruttori di Tipo: Sinonimi e Tipi Algebrici . . . . .              | 18        |
| 4.3.1    | I Sinonimi di Tipo (type) . . . . .                                   | 19        |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| 4.3.2    | I Tipi di Dato Algebrici ( <b>data</b> ) . . . . .                  | 19        |
| 4.4      | Definizione e Derivazione di Istanze . . . . .                      | 19        |
| 4.5      | Gestione del Fallimento: Il tipo Maybe . . . . .                    | 20        |
| 4.6      | Tipi Induttivi e Strutture Dati Ricorsive . . . . .                 | 20        |
| 4.7      | Oltre il Binario: Alberi Ennari ed Espressioni . . . . .            | 21        |
| <b>5</b> | <b>L'onere (e l'onore) della Prova: Dimostrazioni</b>               | <b>22</b> |
| 5.1      | L'Algebra dei Programmi Funzionali . . . . .                        | 22        |
| 5.2      | Induzione Strutturale: L'Esponenziale sui Naturali . . . . .        | 22        |
| 5.3      | L'Induzione sulle Liste e il Lemma di reverse . . . . .             | 23        |
| 5.4      | Funtori, Strictness e Trasformazioni Naturali . . . . .             | 24        |
| 5.5      | Anatomia dei Riduttori: foldr vs foldl . . . . .                    | 24        |
| 5.6      | Complessità Spaziale, Temporale e Lazy Evaluation . . . . .         | 25        |
| <b>6</b> | <b>Ideologia Funzionale in Azione: Comporre, Comporre, Comporre</b> | <b>26</b> |
| 6.1      | Sistemi di Voto: Un approccio Top-Down . . . . .                    | 26        |
| 6.1.1    | Sistemi Elettorali Complessi e Iterazione . . . . .                 | 27        |
| 6.2      | Propaganda e Verità: Il caso del Quicksort . . . . .                | 27        |
| 6.3      | Elaborazione Testi: Il problema delle Common Words . . . . .        | 27        |
| 6.4      | Problemi Combinatori: Esplorare lo Spazio di Ricerca . . . . .      | 28        |
| 6.4.1    | L'Insieme delle Parti (Powerset) . . . . .                          | 28        |
| 6.4.2    | Combinazioni di k elementi . . . . .                                | 29        |
| 6.4.3    | Anagrammi e Permutazioni . . . . .                                  | 29        |
| <b>7</b> | <b>In principio era il Lambda Calcolo</b>                           | <b>30</b> |
| 7.1      | Anima e Corpo della Computazione . . . . .                          | 30        |
| 7.2      | Variabili, $\alpha$ -conversione e Sostituzione formale . . . . .   | 30        |
| 7.3      | Combinatori e Strutture Dati Logiche . . . . .                      | 31        |
| 7.3.1    | Booleani e Condizionali . . . . .                                   | 31        |
| 7.3.2    | Tuple e Proiettori . . . . .                                        | 31        |
| 7.4      | Aritmetica di Church . . . . .                                      | 32        |
| 7.5      | Ricorsione Primitiva e i Suoi Limiti . . . . .                      | 32        |
| 7.6      | Combinatori di Punto Fisso . . . . .                                | 32        |
| 7.7      | Tipaggio in Haskell: Il Sistema Hindley-Milner . . . . .            | 33        |
| 7.7.1    | Regole di Derivazione . . . . .                                     | 33        |
| 7.7.2    | L'Asimmetria del LET e l'Occurs Check . . . . .                     | 34        |
| <b>8</b> | <b>L'Importanza di Essere Lazy: Efficienza ed Espressività</b>      | <b>35</b> |
| 8.1      | La Fusion Law per foldr . . . . .                                   | 35        |
| 8.2      | Induzione su Liste Parziali . . . . .                               | 37        |
| 8.3      | Dentro la Lazy Evaluation: Valori e Forme Normali . . . . .         | 37        |
| 8.3.1    | I Valori in Haskell . . . . .                                       | 38        |
| 8.4      | Il Problema delle Duplicazioni e lo Sharing . . . . .               | 38        |
| 8.5      | I Teoremi di Confluenza e Standardizzazione . . . . .               | 39        |
| 8.6      | Espressività: Funzioni Non Strette e Operatori Logici . . . . .     | 39        |
| 8.6.1    | Il limite della Sequenzialità: Il Parallel OR . . . . .             | 40        |
| <b>9</b> | <b>Funzionali sì, ma efficienti! (Spazio e Tempo)</b>               | <b>41</b> |

|           |                                                                                |           |
|-----------|--------------------------------------------------------------------------------|-----------|
| 9.1       | Il Costo dell'Astrazione . . . . .                                             | 41        |
| 9.2       | Tecnica 1: Tupling (Accoppiamento) . . . . .                                   | 41        |
| 9.2.1     | L'Esempio di Fibonacci . . . . .                                               | 41        |
| 9.2.2     | Zucchero Sintattico: Gli operatori \$ e \$! . . . . .                          | 42        |
| 9.3       | Tecnica 2: Accumulating Parameters . . . . .                                   | 42        |
| 9.4       | Il Lato Oscuro della Laziness: Space Leaks . . . . .                           | 43        |
| 9.4.1     | Forzare la Mano: Il costrutto <code>seq</code> e <code>foldl'</code> . . . . . | 43        |
| 9.5       | Il Tranello delle Tuple . . . . .                                              | 44        |
| 9.6       | Derivazione Formale di Programmi: Il miracolo di <code>scanl</code> . . . . .  | 44        |
| <b>10</b> | <b>L'Infinita Importanza di Essere Lazy: Strutture Dati Infinite</b>           | <b>46</b> |
| 10.1      | Oltre il Finito: Stream e Definizioni Circolari . . . . .                      | 46        |
| 10.2      | Algebra e Co-Algebra dei Tipi Ricorsivi . . . . .                              | 46        |
| 10.3      | Generatori e Zucchero Sintattico . . . . .                                     | 47        |
| 10.4      | Insidie e Produttività: <code>filter</code> e List Comprehension . . . . .     | 47        |
| 10.5      | Complessità Nascoste: Le tre facce di <code>iterate</code> . . . . .           | 48        |
| 10.6      | Fibonacci e il Crivello di Eratostene . . . . .                                | 48        |
| 10.7      | Il Problema degli Hamming Numbers . . . . .                                    | 49        |
| 10.7.1    | Tentativo 1: Il Filtro Inefficiente . . . . .                                  | 49        |
| 10.7.2    | Tentativo 2: Generazione "Artigianale" . . . . .                               | 49        |
| 10.7.3    | Tentativo 3: La Soluzione Definitiva . . . . .                                 | 49        |
| <b>11</b> | <b>Teoria e Pratica degli Stream: Limiti e Generatori Complessi</b>            | <b>51</b> |
| 11.1      | Teoria: Le Liste Infinite come Limiti . . . . .                                | 51        |
| 11.2      | La funzione <code>approx</code> e l'Uguaglianza tra Stream . . . . .           | 52        |
| 11.3      | Dimostrazione per Induzione con <code>approx</code> . . . . .                  | 52        |
| 11.4      | Pratica: Generatori Complessi e i Numeri di Ulam . . . . .                     | 54        |
| 11.4.1    | Gestire le Frequenze: I Multi-insiemi . . . . .                                | 54        |
| 11.4.2    | Costruire l'Algoritmo: <code>nextUlam</code> . . . . .                         | 55        |

# La Babele dei Linguaggi di Programmazione

---

## 1.1 Dagli Algoritmi alla Pratica: Cosa può andare storto?

Che cosa distingue la programmazione dallo studio teorico degli algoritmi? Dal punto di vista teorico, si prende un algoritmo, se ne dimostra la correttezza matematica asintotica e lo si considera completo. Tuttavia, nella pratica, quando si implementa tale algoritmo in un programma, insorgono spesso numerosi problemi:

- **Implementazione errata di operazioni astratte:** la traduzione dalla matematica al codice nasconde insidie.
- **Gestione della memoria ed effetti collaterali (Side-effects):** variabili globali modificate inaspettatamente o puntatori persi.
- **Inefficienza reale:** l'efficienza asintotica non garantisce buone performance se l'uso delle strutture dati dinamiche (come grafi o liste concatenate) non è ottimizzato.

Consideriamo, ad esempio, il problema di verificare se una lista di caratteri è palindroma in C. Un programmatore potrebbe scrivere una funzione che verifica se la lista originaria  $L$  è uguale alla lista invertita restituita da una funzione `reverse(L)`. Affinché questo approccio sia corretto, **reverse non deve modificare** la lista originale  $L$ . Deve allocare e restituire una lista completamente nuova. Se la funzione agisse direttamente sui puntatori di  $L$  invertendoli (un classico effetto collaterale), la verifica di uguaglianza fallirebbe miseramente, portando a risultati imprevedibili.

## 1.2 Computabilità e Astrazione

Da un punto di vista puramente teorico, modelli computazionali minimi come le Macchine di Turing, le Macchine a Registri o il  $\lambda$ -calcolo sono sufficienti per calcolare qualsiasi funzione calcolabile. Tuttavia, per la programmazione pratica, non vogliamo questa minimalità. I linguaggi di programmazione moderni devono fornire:

- **Astrazione del controllo:** costrutti come loop, istruzioni condizionali, funzioni.
- **Astrazione dei dati:** la possibilità di definire tipi di dati complessi, operazioni associate e regole di visibilità.

Come sosteneva Dan Ingalls, *"Un linguaggio deve essere costruito attorno a un piccolo insieme di metafore uniformi e potenti, con una chiara semantica"*.

### 1.3 L'Evoluzione dei Paradigmi di Programmazione

I linguaggi si sono evoluti partendo da un livello puramente hardware fino ad arrivare ad astrazioni logiche e matematiche:

1. **Linguaggi Macchina e Assembler:** Controllo completo su registri e salti. L'assembler ha introdotto etichette simboliche (es. `add %d1 3`), fornendo un primo vitale strato di astrazione che evita di dover ricalcolare indirizzi binari ogni volta che si sposta un'istruzione.
2. **Imperativi (C, Pascal, Fortran):** Un programma è una sequenza di comandi che cambiano lo stato della memoria (assegnamenti). I tipi di dati nascono come astrazione delle celle di memoria per permettere controlli a tempo di compilazione.
3. **Object-Oriented (Java, C++, Smalltalk):** Basati sull'interazione di "oggetti" visti come piccoli automi. L'esecuzione è uno scambio di messaggi tra queste entità che racchiudono stato e comportamento.
4. **Logici (Prolog):** Si descrive una specifica logica, dichiarando *cosa* il programma deve calcolare anziché *come*. Il controllo e la ricerca delle soluzioni sono impliciti nel motore di inferenza.
5. **Funzionali (Haskell, LISP, ML):** Il programma è un insieme di definizioni di funzioni ricorsive. Non esiste un concetto di "memoria" intesa come stato da sovrascrivere, ma solo un ambiente di valutazione. L'esecuzione avviene tramite la riduzione progressiva di espressioni.

### 1.4 Programmazione Funzionale e Haskell

Haskell è considerato il linguaggio funzionale puro per eccellenza. Il suo motto non ufficiale è la **libertà dallo stato**. La programmazione funzionale è affascinante perché enfatizza la valutazione delle funzioni matematiche piuttosto che l'esecuzione di una sequenza di comandi spaziali. Un programma funzionale è semplicemente un'espressione. Computare significa "ridurre" questa espressione, proprio come si faceva alle scuole medie:

$$7 + 5 * (4 + 3) \rightarrow 7 + 5 * 7 \rightarrow 7 + 35 \rightarrow 42$$

Tutta l'informazione necessaria per l'esecuzione risiede all'interno dell'espressione stessa in ogni momento, senza dipendere da stati esterni o variabili invisibili.

### Concetto Chiave: Trasparenza Referenziale

Nei linguaggi imperativi, una variabile rappresenta un indirizzo di memoria il cui contenuto può cambiare nel tempo. Nei linguaggi funzionali, il binding (legame) avviene tra un nome e un **valore immutabile**.

Se consideriamo l'equazione della differenza simmetrica di insiemi:

$$A \otimes B = (A \setminus B) \cup (B \setminus A)$$

Ogni occorrenza di  $A$  (e  $B$ ) denota e denoterà sempre lo stesso esatto insieme. Questa proprietà, per cui un'espressione può essere sempre sostituita con il suo valore valutato senza alterare il comportamento del programma, è detta **trasparenza referenziale**. Elimina totalmente gli effetti collaterali e permette un ragionamento algebrico sul codice.

#### 1.4.1 Tipi, Costruttori e Notazione Infissa

In Haskell, ogni espressione ha un tipo definito matematicamente tramite `::`. Ad esempio, `sin :: Float -> Float`. L'interprete interattivo (GHCi) permette di verificare i tipi in tempo reale con il comando `:type`. Oltre ai tipi predefiniti (`Integer`, `Float`, `Char`), Haskell basa gran parte della gestione dati sui **costruttori di tipo**:

- **Liste**: Sequenze omogenee gestite sequenzialmente, es. `[3, 7, 11] :: [Integer]`. Le stringhe ("tpfi") non sono altro che syntactic sugar per liste di caratteri `['t', 'p', 'f', 'i']`.
- **Tuple**: N-uple di elementi anche disomogenei, utili per restituire risultati multipli da una funzione, es. `(3, 'c') :: (Integer, Char)`.

È interessante notare come gli operatori come `+` o `++` siano a tutti gli effetti delle funzioni binarie. Possono essere usati in forma prefissa con le parentesi: `(+) 3 5`. Similmente, una funzione normale può essere usata come operatore infisso racchiudendola tra backtick: `5 `div` 3`. Inoltre, è possibile l'**applicazione parziale**: se definiamo `sum3 = (+) 3`, stiamo passando un solo argomento a una funzione che ne richiede due, ottenendo in ritorno una nuova funzione unaria di tipo `Integer -> Integer`.

### Concetto Chiave: Polimorfismo e Classi (Bounded Polymorphism)

Il tipo dell'operatore di concatenazione (`++`) è `[a] -> [a] -> [a]`. La variabile `a` è una **variabile di tipo** (quantificata universalmente:  $\forall a$ ). Questo garantisce che la funzione lavori con liste di qualsiasi tipo, a patto che tutti gli argomenti e il risultato siano esattamente dello stesso tipo (es. non si possono fondere liste di interi con liste di caratteri).

Quando il polimorfismo deve essere ristretto a specifici comportamenti, entrano in gioco le **Classi** (simili alle interfacce in Java). Ad esempio, la firma della funzione di ordinamento `sort` è: `sort :: (Ord a) => [a] -> [a]`. Significa: "Per ogni tipo `a` che appartiene alla classe `Ord` (ovvero su cui è definita una relazione di ordine), prendi una lista di `a` e restituisci una lista di `a` ordinata".

### 1.4.2 Ordine Superiore e Composizionalità

La forza dell'astrazione funzionale risiede nell'usare **funzionali** (funzioni di ordine superiore) che accettano o restituiscono altre funzioni:

- `map :: (a -> b) -> [a] -> [b]`: Mappa una funzione unaria su tutti gli elementi di una lista.
- `zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]`: Combina due liste applicando una funzione binaria.
- `(.) :: (b -> c) -> (a -> b) -> a -> c`: Operatore di composizione funzionale, analogo al  $\circ$  matematico.

L'assenza di stato e la composizione permettono di dimostrare l'equivalenza dei programmi tramite il calcolo algebrico. Ad esempio, è banale dimostrare equazionalmente che applicare una composizione di funzioni a ogni elemento di una lista equivale a mappare prima l'una e poi l'altra:

$$\text{map } (f \cdot g) = (\text{map } f) \cdot (\text{map } g)$$

#### Concetto Chiave: Lazy Evaluation (Valutazione Pigra)

Haskell valuta le espressioni in modalità *lazy*. A differenza dei linguaggi imperativi (che valutano prima gli argomenti e poi chiamano la funzione), Haskell posticipa l'esecuzione finché il risultato non è strettamente richiesto per proseguire il calcolo globale.

Questo paradigma permette, ad esempio, di dichiarare strutture dati infinite senza mandare la macchina in stallo (out of memory). Definendo `ones = 1 : ones`, creiamo una lista infinita di 1. Se creiamo una funzione che estrae il primo elemento, `myHead (x:xs) = x`, ed eseguiamo `myHead (map (*2) ones)`, Haskell non proverà a moltiplicare per 2 infiniti numeri. Valuterà solo la testa necessaria:  $1 \cdot 2 = 2$ , e terminerà immediatamente restituendo il risultato.

### 1.4.3 Il calcolo dei programmi: Valutazione Ricorsiva e undefined

A livello puramente meccanico, l'interprete applica sostituzioni testuali (pattern matching) ed espansioni fino a raggiungere un caso base. Se definiamo il fattoriale tramite ricorsione con un caso base per '0' e un caso generico per 'n':

```
1 fatt' (0, f) = f
2 fatt' (n, f) = fatt' (n-1, f*n)
3 fatt n = fatt' (n, 1)
```

La valutazione di `fatt 3` procede esattamente come una riduzione matematica lazy:

1. `fatt 3` viene sostituito dalla sua definizione: `fatt' (3, 1)`
2. Applica il pattern (n): `fatt' (3-1, 3*1)`
3. Prima di proseguire con `fatt'`, deve valutare l'espressione aritmetica minima necessaria per il matching di sinistra: `fatt' (2, 3*1)`



4. Applica di nuovo il pattern:  $\text{fatt}' (2-1, 2*3*1) \rightarrow \text{fatt}' (1, 2*3*1)$
5. Ancora:  $\text{fatt}' (1-1, 1*2*3*1) \rightarrow \text{fatt}' (0, 1*2*3*1)$
6. Ora il pattern corrisponde al caso base  $(0, f)$ . La funzione restituisce direttamente il parametro  $f$ : restituisce  $1*2*3*1$ .
7. Solo in questa fase finale valuta i prodotti in sospenso:  $1 \cdot 2 \cdot 3 \cdot 1 = 6$ .

Infine, merita menzione l'esistenza in Haskell del valore speciale `undefined`. Questo costrutto possiede un tipo apparentemente assurdo `undefined :: a` ( $\forall a$ , compatibile con *qualsiasi* tipo: da interi, a liste, a funzioni complesse). La sua firma riflette matematicamente l'assenza di informazioni semantiche ed è utilizzata come rappresentazione canonica per denotare computazioni inconsistenti, fallimenti previsti o cicli di non-terminazione (loop infiniti).

## 2

# Quegli Strani Tipi (Polimorfi) e Funzioni

## 2.1 Type Inference e la Funzione Identità

In Haskell possiamo definire le funzioni dando un nome a un'espressione oppure utilizzando le lambda-espressioni (funzioni anonime). Un esempio fondamentale è la funzione identità, definibile come  $i\ x = x$  oppure  $i' = \lambda x \rightarrow x$ .

Quando scriviamo questa funzione, non forniamo al compilatore alcuna informazione sul tipo dei parametri. È il compilatore stesso che interviene tramite il processo di **type inference** (inferenza dei tipi), calcolando il *tipo principale*. Il tipo di  $i$  risulta essere  $t \rightarrow t$ . Tutti gli altri tipi corretti per l'identità non sono altro che istanze di questo tipo generale, ottenute sostituendo la variabile di tipo  $t$  con un tipo specifico (ad esempio  $\text{Int} \rightarrow \text{Int}$ ).

### Static Type Safety e Polimorfismo Universale

Haskell è *statically type-safe*: se un programma supera il controllo dei tipi durante la compilazione, si ha la garanzia matematica che non si verificheranno errori di tipo a runtime. La funzione identità mostra la potenza del **polimorfismo**: è definita per qualunque tipo esistente. Infatti, l'identità è l'unica funzione che può vantare il tipo universale  $\forall a. a \rightarrow a$ .

È possibile tradurre agevolmente i combinatori base del  $\lambda$ -calcolo:

- **Combinatore K** (proietta il primo argomento ignorando il secondo):  $k\ x\ y = x$ . Il suo tipo è  $t_1 \rightarrow t_2 \rightarrow t_1$ .
- **Combinatore S** (applicazione distribuita):  $s\ x\ y\ z = x\ z\ (y\ z)$ .

## 2.2 Termini non Tipabili, Non-Terminazione e undefined

Non tutte le lambda-espressioni valide a livello teorico possono essere tipate dal sistema di Haskell. L'esempio più famoso è il combinator  $\Omega = \lambda x \rightarrow x\ x$ . Se tentiamo di definirlo, GHCi restituisce un errore di *occurs check*: il sistema si accorge che dovrebbe tipare la variabile  $x$  simultaneamente con un tipo base  $t_1$  e con un tipo freccia  $t_1 \rightarrow t$ , creando un tipo infinito impossibile da risolvere. Di conseguenza, tutti i lambda-termini tipabili in Haskell sono garantiti per terminare.

Si possono comunque definire funzioni ricorsive che non terminano, ad esempio:  $\omega' x = \omega' x$ . Il compilatore inferisce per questa funzione il tipo  $t \rightarrow t_1$ . Ma quale funzione può accettare un parametro di un qualsiasi tipo e restituire magicamente un risultato di un altro qualsiasi tipo scorrelato? L'unica spiegazione è che la funzione *non restituisca mai nulla* perché entra in un loop infinito. In programmazione funzionale, la semantica di un'equazione ricorsiva è identificabile come il *punto fisso* della trasformazione che la definisce.

### L'oggetto indefinito: undefined

In Haskell esiste una costante predefinita chiamata **undefined**. Se ne esploriamo il tipo, otteniamo  $\forall a.a$ . Questo tipo è "inconsistente" da un punto di vista logico: possiede il tipo più generale in assoluto, e dunque appartiene a tutti i tipi contemporaneamente. È la rappresentazione canonica di computazioni indefinite, fallimenti previsti o loop infiniti.

## 2.3 Costrutti, Pattern Matching e Tuple

Oltre ai tipi predefiniti matematici (*Int*, *Integer*, *Float*, *Double*) e testuali (*Char*, *String*), la logica di controllo si basa fortemente sul pattern matching. Nei linguaggi funzionali, il ramo *else* del costrutto *if* non è opzionale: un *if* è un'espressione che deve sempre restituire un valore valutabile. Un'alternativa più pulita sono le **espressioni guardate** ( $| \text{condizione} = \text{risultato}$ ), le quali vengono valutate sequenzialmente.

Il Vero Programmatore Haskell (VPH) evita, quando possibile, l'uso massiccio di costrutti **if-then-else**, preferendo di gran lunga il **Pattern Matching**. Questa tecnica permette di definire il comportamento di una funzione analizzando e smontando direttamente la forma sintattica (il "pattern") dei parametri in ingresso.

L'ordine in cui si scrivono le clausole è assolutamente cruciale: Haskell le analizza dall'alto verso il basso ed esegue *solo la prima* che fa "match" col parametro fornito. Se la clausola più generale (quella che cattura qualsiasi valore) viene messa per prima, i casi specifici successivi non verranno mai raggiunti.

### 2.3.1 L'evoluzione del NOT e il parametro anonimo

Consideriamo la funzione di negazione logica. Un primo approccio con pattern matching esplora esplicitamente tutti i casi:

```
1 myNot ' False = True
2 myNot ' True  = False
```

Tuttavia, siccome l'ordine conta, se il parametro non è `False`, sarà per forza `True`. Potremmo quindi generalizzare la seconda clausola usando una variabile `x`:

```
1 myNot '' False = True
2 myNot '' x     = False
```

In questo caso, `x` fa matching con tutto ciò che non è `False` (quindi `True`). Ma c'è un problema di stile: il nome della variabile `x` non viene mai utilizzato nel lato destro dell'equazione (il risultato). Quando un parametro non è utile al calcolo, il VPH usa l'**underscore** (`_`), ovvero il parametro anonimo (o "don't care"):

```
1 myNot '' False = True
2 myNot '' _     = False
```

### 2.3.2 Funzioni Binarie e il Divieto di Unificazione

Il pattern matching si estende elegantemente alle funzioni con più parametri, come l'AND e l'OR logico:

```
1 myAnd True True = True
2 myAnd _ _       = False
3
4 myOr False False = False
5 myOr _ _         = True
```

Esiste però un **limite invalicabile**: il pattern matching in Haskell **non permette di fare unificazioni**. Questo significa che non si può usare la stessa variabile più volte nello stesso pattern per imporre che due parametri debbano essere identici. Se provassimo a scrivere lo XOR logico in questo modo:

```
1 -- ERRORE: Il Pattern matching non permette unificazioni!
2 xor x x = False
3 xor _ _ = True
```

Il compilatore restituirebbe l'errore `Conflicting definitions for 'x'`. Per imporre condizioni di uguaglianza o disuguaglianza tra i parametri, è obbligatorio ricorrere alle **espressioni guardate**:

```
1 xor x y
2   | x /= y     = True
3   | otherwise  = False
```

### 2.3.3 Pattern Matching sui Numeri Naturali

L'applicazione del pattern matching sui numeri naturali rende la scrittura delle funzioni matematiche ricorsive estremamente naturale, quasi identica alla loro definizione formale. Qui l'importanza di mettere i casi base in cima diventa ancora più evidente.

```

1 -- Il caso base deve obbligatoriamente precedere quello generale
2 fatt 0 = 1
3 fatt n = n * fatt (n - 1)
4
5 -- La famigerata (e inefficiente) funzione di Fibonacci
6 fib 0 = 0
7 fib 1 = 1
8 fib n = fib (n - 1) + fib (n - 2)

```

### 2.3.4 Decomposizione di Tuple e Lambda

Infine, il pattern matching funziona perfettamente anche sui valori strutturati come le Tuple. Funziona da vero e proprio "distruttore", permettendo di estrarne i componenti interni direttamente nella firma della funzione. Scrivendo le funzioni predefinite `fst` e `snd`:

```

1 myFst (x, y) = x
2 mySnd (x, y) = y

```

Una caratteristica potente e meno nota è che il pattern matching può essere utilizzato *direttamente all'interno delle lambda-espressioni* (le funzioni anonime). Possiamo infatti scrivere le proiezioni della tupla senza assegnare loro un nome:

```

1 myFst' = \ (x, y) -> x
2 mySnd' = \ (x, y) -> y

```

Questo risulta utilissimo quando si devono passare funzioni anonime "al volo" a funzionali di ordine superiore come `map` o `filter`, decomponendo le strutture dati in ingresso senza dover definire funzioni d'appoggio esterne.

L'applicazione di una funzione in Haskell si scrive separando gli argomenti con spazi ( $f\ x\ y$ ). La notazione tipica imperativa  $f(x, y)$  significa invece applicare la funzione a un unico argomento strutturato: una **Tupla**. Le tuple possono contenere tipi eterogenei e su di esse è possibile fare pattern matching diretto, persino all'interno delle lambda-espressioni.

Per dichiarazioni locali, si utilizzano due costrutti:

1. **let ... in**: È una vera e propria espressione. Molto utile per il *tupling*, ovvero la decomposizione di tuple restituite da chiamate ricorsive (ad esempio, per un calcolo di Fibonacci lineare).
2. **where**: Non è un'espressione autonoma, ma una clausola che qualifica e definisce variabili utilizzate nella parte destra di un'equazione principale, rendendo il codice estremamente leggibile e simile alla notazione matematica.

## 2.4 Programmazione su Liste

Nella tradizione dei linguaggi funzionali, fin dai tempi del LISP, la lista è la struttura dati fondamentale. In Haskell, le liste sono omogenee (possono contenere elementi di un solo tipo) e sono costruite a partire da due elementi base:

- La lista vuota: `[]`, che possiede il tipo polimorfo `[t]`.
- L'operatore di costruzione (`cons`): `:`, che inserisce un elemento in testa a una lista. Il suo tipo è `t -> [t] -> [t]`.

La scrittura `[1, 2, 3]` è semplicemente "zucchero sintattico" fornito dal linguaggio per nascondere la vera struttura sottostante, che è `1 : (2 : (3 : []))`. Allo stesso modo, le stringhe non sono altro che liste di caratteri: la stringa `"tpfi"` equivale a `['t', 'p', 'f', 'i']`.

### 2.4.1 Pattern Matching: Testa, Coda e Parentesi

Quasi tutte le funzioni sulle liste in Haskell vengono scritte separando la lista nelle sue due componenti fondamentali tramite il pattern `(x:xs)`, dove `x` rappresenta la **testa** (il primo elemento) e `xs` rappresenta la **coda** (il resto della lista). Gli Haskelloti usano spesso il suffisso `s` (plurale) per indicare che una variabile contiene una lista.

**Attenzione alle parentesi:** In Haskell, l'applicazione di funzione ha la precedenza assoluta su qualsiasi altro operatore. Se scrivessimo `f x:xs` senza parentesi, l'interprete lo leggerebbe come `(f x) : xs`, portando a inevitabili errori di tipo. È obbligatorio scrivere `f (x:xs)`.

Vediamo come definire gli estrattori di base, `head` e `tail`:

```
1 myHead (x:_) = x
2 myTail (_:xs) = xs
```

Cosa succede se passiamo una lista vuota `[]` a queste funzioni? Poiché non abbiamo fornito una clausola per il caso base `[]`, l'interprete genererà un'eccezione a runtime (`Non-exhaustive patterns`). In Haskell questo è spesso un comportamento voluto: estrarre la testa da una lista vuota è un errore logico che deve interrompere l'esecuzione.

### 2.4.2 Ispezione ed Esplorazione: `null`, `last` e `init`

Esistono diverse funzioni predefinite per ispezionare le liste senza distruggerle.

- `null`: Verifica se una lista è vuota (restituisce `True` o `False`).
- `last`: Restituisce l'ultimo elemento (scorre tutta la lista).
- `init`: Restituisce tutta la lista tranne l'ultimo elemento.

### Pillole di Laziness: Il comportamento di null

La valutazione pigra (lazy evaluation) mostra i suoi effetti anche su funzioni semplici come `null`. Questa funzione è considerata "stretta" (strict) solo nella misura in cui ha bisogno di guardare il primo nodo per capire la struttura. Se eseguiamo `null undefined`, il programma va in crash (`Exception: Prelude.undefined`). Ma se eseguiamo `null (undefined : undefined)`, il risultato sarà `False`! Haskell non ha bisogno di valutare il contenuto della lista (che è indefinito); gli basta vedere che esiste un costruttore `:` per dedurre matematicamente che la lista non è vuota.

Vediamo come si implementerebbero `last` e `init` tramite ricorsione:

```
1 myLast [x] = x
2 myLast (_:xs) = myLast xs
3
4 myInit [x] = []
5 myInit (x:xs) = x : myInit xs
```

### 2.4.3 L'operatore di Concatenazione (++)

La giustapposizione di due liste si effettua con l'operatore `++`. La sua firma è `[a] -> [a] -> [a]`. È un operatore associativo e la sua definizione ricorsiva si basa esclusivamente sullo smontaggio del primo parametro:

```
1 [] ++ ys = ys
2 (x:xs) ++ ys = x : (xs ++ ys)
```

Anche qui, la laziness crea scenari interessanti:

- `undefined ++ [1, 2] → Crash`. La funzione deve fare pattern matching sul primo parametro, che essendo `undefined` fa esplodere il programma.
- `[1, 2] ++ undefined → Risultato: [1, 2, *** Exception]`. Haskell riesce a costruire la prima parte della lista, stampando 1 e 2, prima di scontrarsi con l'indefinito al momento di agganciare la coda.

### 2.4.4 Estrazione e Scomposizione (take, drop, splitAt, !!)

Haskell fornisce funzioni per estrarre porzioni di lista in base a un indice numerico:

- `xs !! n`: Estrae l'elemento in posizione `n` (partendo da 0). Se l'indice è troppo grande, genera un'eccezione (`index too large`).
- `take n xs`: Prende i primi `n` elementi di una lista. Se `n` è maggiore della lunghezza, si ferma senza errori restituendo l'intera lista.
- `drop n xs`: Rimuove i primi `n` elementi, restituendo il resto.
- `splitAt n xs`: Restituisce una tupla contenente due liste: il risultato di `take n xs` e il risultato di `drop n xs`.

### Concetto Chiave: L'As-Pattern per le ottimizzazioni

Quando manipoliamo liste complesse, capita spesso di fare pattern matching per estrarre la testa e la coda, ma allo stesso tempo avere bisogno di passare alla successiva chiamata ricorsiva *l'intera lista* intatta. Ricostruire la lista scrivendo `x:xs` richiede l'allocazione di un nuovo nodo `cons` in memoria. Per evitare questo spreco, Haskell offre l'**as-pattern**, indicato con il simbolo `@`. Scrivendo `xs@(x:txs)`, stiamo dicendo: "smonta la lista in `x` e `txs`, ma memorizza un riferimento all'intera lista originale nella variabile `xs`".

Un esempio classico dell'uso dell'as-pattern è la funzione `merge`, che fonde due liste ordinate (passaggio fondamentale del *Merge Sort*):

```
1 merge [] ys = ys
2 merge xs [] = xs
3 merge xs@(x:txs) ys@(y:tys)
4   | x <= y      = x : merge txs ys
5   | otherwise   = y : merge xs tys
```

Senza l'as-pattern, l'ultima riga avrebbe dovuto essere `y : merge (x:txs) tys`, costringendo il computer a ricostruire inutilmente una porzione di lista già esistente.

#### 2.4.5 Liste Annidate: `zip` e `concat`

Le liste possono essere annidate a piacimento (es. `[[1,2], [3,4]]`). Tra le funzioni strutturali più importanti troviamo:

- `zip`: Prende due liste e le "cuce" insieme, creando una singola lista di tuple. Si ferma automaticamente non appena la lista più corta si esaurisce.
- `concat`: "Scioglie" una lista di liste in un'unica lista piatta, giustapponendo tutti i valori interni in ordine.

```
1 myZip (x:xs) (y:ys) = (x, y) : myZip xs ys
2 myZip _ _ = [] -- Si ferma se una delle due è vuota
3
4 myConcat (xs:xss) = xs ++ myConcat xss
5 myConcat [] = []
```

Come si può notare, `myConcat` ha la stessa identica struttura ricorsiva della funzione `sum`. L'unica differenza è che, invece di usare `0` e `+`, usa la lista vuota `[]` e l'operatore `++`. Questa forte somiglianza strutturale è il preludio al concetto di *funzionale di ordine superiore* (come `foldr`), che astrae completamente questi pattern ricorsivi.

# Verso un Ordine Superiore: Funzionali e Applicazioni

## 3.1 Program Calculation: Valutazione passo-passo

I programmi Haskell si riducono esattamente come le espressioni algebriche che si risolvono alle scuole medie. Per capire a fondo la computazione, è utile tracciare l'esecuzione a mano, un processo noto come *Program Calculation*. Prendiamo ad esempio l'operatore di concatenazione ++ (append), le cui clausole definitorie sono:

```
1 [] ++ ys = ys          -- (1)
2 (x:xs) ++ ys = x : (xs ++ ys)  -- (2)
```

Se valutiamo `[1,2] ++ [3,4,5]`, il calcolo procede così:

1. Prima espansione sintattica: `(1:(2:[])) ++ (3:(4:(5:[])))`
2. Applichiamo la clausola (2): `1 : ((2:[]) ++ (3:(4:(5:[]))))`
3. Applichiamo di nuovo la (2): `1 : (2 : ([] ++ (3:(4:(5:[])))))`
4. Ora il primo argomento è vuoto, applichiamo la clausola (1): `1 : (2 : (3:(4:(5:[]))))`
5. Tornando alla notazione compatta, il risultato è `[1, 2, 3, 4, 5]`.

## 3.2 Zucchero Sintattico: Le Enumerazioni

Per evitare di scrivere manualmente liste sequenziali, Haskell fornisce delle comode notazioni chiamate **enumerazioni**:

- `[m..n]`: genera la lista finita da  $m$  a  $n$ . Esempio: `[1..5]` genera `[1,2,3,4,5]`.
- `[m..]`: genera una lista *infinita* a partire da  $m$ . Grazie alla lazy evaluation, possiamo scrivere `[0..]` per indicare tutti i numeri naturali, senza bloccare la macchina.
- `[m, n..p]`: genera una lista finita in cui il "passo" (la distanza tra gli elementi) è definito dalla differenza tra  $n$  e  $m$ . Ad esempio, `[1, 3..11]` restituisce i dispari `[1,3,5,7,9,11]`.
- `[m, n..]`: come sopra, ma genera una lista infinita (es. `[1, 3..]` genera tutti i dispari esistenti).

Questa notazione funziona su tutti i tipi enumerabili. Possiamo scrivere `['a'..'z']` per ottenere l'intero alfabeto minuscolo. Tuttavia, non si possono generare sequenze non



puramente aritmetiche: `[1, 1, 2, 3, 5..]` è sintatticamente scorretto e non genererà la sequenza di Fibonacci!

### 3.3 Curryficazione e Manipolazione degli Argomenti

Il concetto di curryficazione (dal nome del logico Haskell Curry) è il pilastro della composizionalità funzionale. Invece di definire funzioni che accettano più parametri incapsulati in una singola tupla, si definiscono funzioni che accettano un parametro alla volta, restituendo per ciascuno una nuova funzione che attende il parametro successivo.

#### Concetto Chiave: Isomorfismo della Curryficazione

Da un punto di vista degli insiemi, esiste un isomorfismo garantito (basta fare attenzione all'associatività delle parentesi):

$$A \times B \rightarrow C \cong A \rightarrow (B \rightarrow C)$$

Il grande vantaggio delle versioni curryficate è l'applicazione parziale. Passando un solo argomento a una funzione binaria, otteniamo in ritorno una funzione unaria specializzata, perfettamente compatibile con i funzionali di ordine superiore.

Haskell fornisce tre piccoli funzionali predefiniti che aiutano a convertire e manipolare la firma delle funzioni:

```
1 -- Trasforma una funzione su tupla in una funzione curryficata
2 myCurry f x y = f (x,y)
3 -- Tipo: ((a, b) -> c) -> a -> b -> c
4
5 -- Trasforma una funzione curryficata in una che accetta una tupla
6 myUncurry f (x,y) = f x y
7 -- Tipo: (a -> b -> c) -> (a, b) -> c
8
9 -- Inverte l'ordine dei due parametri di una funzione
10 myFlip f a b = f b a
11 -- Tipo: (a -> b -> c) -> b -> a -> c
```

L'operatore `flip` è incredibilmente utile. Se vogliamo inserire l'elemento 1 in fondo a una lista `[2,3]`, possiamo sfruttare il `cons (:) invertito`: `flip (:) [2,3] 1` produrrà `[1,2,3]`.

### 3.4 Schemi di Programmazione: `map` e `filter`

Le operazioni sulle liste seguono spesso schemi ricorrenti che vengono astratti in funzioni di ordine superiore (funzionali). La funzione `map` applica una trasformazione a tutti gli elementi di una lista. Gode di proprietà algebriche notevoli, come `map id = id` (mappare l'identità restituisce la lista invariata) e `map (f . g) = map f . map g` (mappare la composizione di funzioni equivale a comporre le mappe).

La funzione `filter` seleziona invece i valori che soddisfano un certo predicato (una funzione che restituisce un `Bool`). Un modo molto istruttivo per definire `filter` è esprimerlo in termini di `map` e `concat`:

```

1 -- Funzione di test: restituisce [x] se vero, lista vuota [] se falso
2 test p x = if p x then [x] else []
3
4 -- filter definito componendo map e concat
5 filter p xs = concat (map (test p) xs)

```

Se calcoliamo `filter odd [15, 44, 81]` (dove `odd` verifica se un numero è dispari), `map` genera `[[15], [], [81]]`. L'applicazione successiva di `concat` schiaccia la lista rimuovendo le sottoliste vuote, restituendo `[15, 81]`.

### 3.5 List Comprehension: La Matematica in Haskell

Un costrutto sintattico estremamente amato per manipolare le liste è la **List Comprehension**, che ricalca la definizione matematica degli insiemi (set comprehension), come  $\{x \in A \mid P(x)\}$ . In Haskell si scrive:

```

1 > [x^2 | x <- [1..5]]
2 [1, 4, 9, 16, 25]

```

Si possono aggiungere dei predicati (guards) che fungono da filtro. Ad esempio, per trovare tutti i fattori di un numero  $n$ :

```

1 factors n = [x | x <- [1..n], n `mod` x == 0]

```

La vera potenza si esprime con generatori multipli, che creano il prodotto cartesiano di più liste. L'ordine in cui si scrivono i generatori è fondamentale (l'ultimo generatore varia più rapidamente):

```

1 > [(x,y) | x <- [1,2,3], y <- [4,5]]
2 [(1,4), (1,5), (2,4), (2,5), (3,4), (3,5)]

```

### 3.6 Il Prodotto Scalare: Un Trionfo dell'Ordine Superiore

Nel 1978, John Backus scrisse un famoso articolo esaltando la programmazione funzionale. Usò il calcolo del prodotto scalare tra due vettori come esempio principe per dimostrare come sbarazzarsi di accumulatori e indici  $i$ . In Haskell, l'approccio classico unisce `zip`, `map` e `sum`:

```

1 ps1 xs ys = sum (map (uncurry (*)) (zip xs ys))

```

Se valutiamo `ps1 [5,4,9] [1,10,100]`:

1. `zip` crea la lista di coppie: `[(5,1), (4,10), (9,100)]`. Se una lista è infinita, `zip` si ferma automaticamente (grazie alla laziness) non appena finisce la lista più corta.
2. `map` applica la moltiplicazione. Poiché `(*)` richiede due parametri separati, si usa `uncurry` per adattarla a ricevere la tupla: otteniamo `[5, 40, 900]`.
3. `sum` somma i valori restituendo 945.

Per semplificare ulteriormente, si può usare `zipWith`, un funzionale formidabile che incapsula il comportamento di `map` e `zip`, applicando direttamente una funzione binaria agli elementi omologhi di due liste:

```

1 zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
2 ps3 xs ys = sum (zipWith (*) xs ys)

```

### 3.7 Esistenziale ed Universale: any, all e la Ricerca su Liste

Le funzioni logiche "esiste almeno un elemento che..." e "tutti gli elementi soddisfano..." si traducono in **any** e **all**. Possono essere elegantemente definite combinando **map** e le riduzioni booleane predefinite **or** / **and**:

```

1 myAny p = or . map p
2 myAll p = and . map p

```

Cosa succede se vogliamo implementare una **ricerca sequenziale** per trovare l'indice di un elemento in una lista (restituendo -1 se non trovato)? Un programmatore imperativo userebbe un contatore. Il Vero Programmatore Haskell (VPH) astrae il problema creando una lista di tuple (indice, valore), filtrando quelle che corrispondono al valore cercato ed estraendone l'indice.

```

1 findVPH v xs =
2   (fst . head) (filter (\(x,y) -> v == y) (zip [0..] xs) ++ [(-1,v)])

```

La genialità sta nell'appendere in fondo la tupla "fittizia" `[(-1,v)]`: se il **filter** scarta tutto perché l'elemento non c'è, **head** troverà comunque `(-1,v)`, ed estraendo il primo elemento con **fst** restituirà in modo sicuro -1.

### 3.8 I Funzionali di Riduzione: foldr e foldr1

Analizzando le definizioni di funzioni come **sum**, **length** o **and**, si nota che condividono la medesima ossatura ricorsiva.

#### Concetto Chiave: La potenza di foldr

Il funzionale **foldr** (fold right) astrae la ricorsione sulle liste. Data una funzione binaria  $\oplus$  e un valore base  $v$ , il comportamento è:

$$\text{foldr } (\oplus) v [x_0, x_1, \dots, x_n] = x_0 \oplus (x_1 \oplus (\dots (x_n \oplus v) \dots))$$

**foldr** sostituisce virtualmente ogni costruttore di lista `(:)` con la funzione  $\oplus$ , e la lista vuota `[]` con il valore base  $v$ , "ripiegando" i risultati da destra verso sinistra.

Le funzioni base diventano *one-liner*:

```

1 mySum      = foldr (+) 0
2 myAnd      = foldr (&&) True
3 myLength   = foldr (\x -> (+1)) 0

```

Per operazioni come la ricerca del minimo (**min**) o del massimo (**max**), non esiste un "elemento neutro" scontato (come lo zero per la somma) da usare su lista vuota senza alterare il calcolo. In questi casi si usa **foldr1**, che utilizza l'ultimo elemento della lista stessa come caso base. L'unica controindicazione è che **foldr1** andrà in errore se applicata a una lista completamente vuota.

### 3.9 Applicazioni Avanzate: Matrici e Verifica di Ordinamento

Possiamo sfruttare questi funzionali astratti per operazioni complesse, minimizzando il codice. Per verificare se una lista è **ordinata**, basta sovrapporla con la sua coda (traslata di 1) e verificare che ogni elemento sia minore o uguale al successivo:

```
1 ordinataVPH xs = and (zipWith (<=) xs (tail xs))
```

Anche la **moltiplicazione tra matrici** diventa un capolavoro di sintesi. Rappresentando una matrice come una lista di liste (per righe), la moltiplicazione consiste nel fare il prodotto scalare tra le righe della prima matrice e le colonne della seconda. Scriviamo prima una funzione per calcolare la trasposta (che inverte righe in colonne), e poi usiamo `map`:

```
1 trasposta [] = repeat []
2 trasposta (xs:xss) = zipWith (:) xs (trasposta xss)
3
4 prodMat a b =
5   map (\riga -> map (prodottoScalare riga) (trasposta b)) a
```

Queste due righe di codice sostituiscono i classici tre cicli `for` annidati: per ogni riga di `a`, mappiamo su tutta la `trasposta b` la funzione `prodottoScalare` (che grazie alla curryficazione e all'astrazione ha già "incamerato" la riga corrente come primo parametro). Un vero miracolo dell'ordine superiore.

# Tipi Strani, ma di Classe

## 4.1 Classi e Subtyping: Il Polimorfismo Vincolato

Finora abbiamo esplorato funzioni fortemente tipizzate o completamente polimorfe (come l'identità). Esiste però una via di mezzo fondamentale in Haskell: funzioni che operano su un insieme vasto ma specifico di tipi, a patto che questi condividano certe operazioni.

Prendiamo in esame la funzione per il calcolo del Massimo Comune Divisore (MCD) tramite l'algoritmo delle sottrazioni ripetute di Euclide:

```
1 mcd x y
2   | x == y      = x
3   | x < y       = mcd (y - x) x
4   | otherwise  = mcd (x - y) y
```

Potremmo aspettarci che l'interprete inferisca un tipo limitato agli interi, come `Integer -> Integer -> Integer`. Invece, GHCi calcola: `mcd :: (Ord a, Num a) => a -> a -> a`

### Concetto Chiave: Il significato di `=>` e le Classi

Tutto ciò che si trova a sinistra della freccia `=>` rappresenta il **vincolo di classe** (class constraint). Il tipo `a` non può essere un tipo qualsiasi, ma deve contemporaneamente appartenere alla classe `Ord` (perché usiamo `<` e `==`) e alla classe `Num` (perché usiamo la sottrazione `-`). Le Type Classes di Haskell sono simili alle *Interfacce* in Java: impongono che un tipo implementi un insieme specifico di funzioni o operatori per essere considerato un'istanza valida di quella classe.

Questa logica si applica anche alle costanti numeriche. Se interroghiamo l'interprete sul tipo del numero `3`, otterremo `Num a => a`. Il `3` è polimorfo: può comportarsi come un intero, un decimale o un numero a precisione arbitraria, ma non potrà mai assumere il ruolo di un carattere o di un booleano.

A un livello più granulare, le operazioni di divisione ci mostrano altre sfaccettature: l'operatore `(/)` per la divisione reale richiede la classe `Fractional` (implementata da `Float` e `Double`), mentre l'operatore `div` per la divisione intera richiede la classe `Integral` (implementata da `Int` e `Integer`).

## 4.2 La Gerarchia delle Classi Predefinite

Le classi non esistono in un vuoto isolato, ma formano una rigida gerarchia algebrica.

### 4.2.1 La Classe Eq (Uguaglianza)

La classe `Eq` racchiude i tipi su cui ha senso chiedersi se due valori siano uguali (`==`) o diversi (`/=`). Tutti i tipi base sono istanze di `Eq`. Liste e tuple diventano istanze di `Eq` "gratis", a patto che i tipi che contengono siano a loro volta uguagliabili.

```
1 class Eq a where
2     (==), (/=) :: a -> a -> Bool
3     -- Haskell permette di inserire codice di default nella classe:
4     x /= y = not (x == y)
```

Fornendo l'implementazione di default per `/=`, il programmatore che vuole rendere un nuovo tipo istanza di `Eq` è obbligato a scrivere solamente il codice per `==`.

### 4.2.2 La Classe Ord (Ordinamento)

Gli insiemi parzialmente ordinati appartengono alla classe `Ord`, la quale definisce operatori come `<`, `<=`, `>`, `min` e `max`. La caratteristica vitale di `Ord` è che essa **estende** `Eq`:

```
1 class Eq a => Ord a where
2     (<), (<=), (>=), (>) :: a -> a -> Bool
3     min, max :: a -> a -> a
```

Non può esistere un tipo ordinabile che non sia prima uguagliabile. L'ordine per strutture come liste e tuple è definito *lessicograficamente* (es. `[1,2] < [1,2,3]` valuta a `True`).

### 4.2.3 Le Classi Num, Show e Read

La classe numerica più generica, `Num`, richiede a sua volta che i numeri siano sia uguagliabili (`Eq`) sia stampabili a schermo (`Show`). Le classi per la gestione di input/output testuale sono:

- **Show**: fornisce `show :: a -> String`, convertendo un valore nel suo equivalente testuale.
- **Read**: fornisce `read :: String -> a`, effettuando il parsing inverso per ricostruire un valore da una stringa.

Quando l'interprete valuta un'espressione, cerca sempre di stamparla usando `show`. Questo spiega perché, se valutiamo a terminale una funzione (come `\x -> x`), Haskell va in crash (`No instance for Show`): le funzioni, essendo concetti computazionali astratti, non possiedono di default una rappresentazione in stringa.

## 4.3 Costruttori di Tipo: Sinonimi e Tipi Algebrici

Haskell ci permette di arricchire l'universo dei tipi in due modi distinti: creando sinonimi o coniando nuovi tipi da zero.

### 4.3.1 I Sinonimi di Tipo (type)

La parola chiave `type` introduce un *alias*. Non crea nulla di strutturalmente nuovo, ma migliora enormemente la leggibilità del codice, agendo come una macro.

```
1 type Casella = (Int, Int)
2 type Move = (Int, Int)
3 type Pair a b = (a, b)
```

Essendo solo sinonimi di tuple, Haskell non applicherà restrizioni di tipo se scambiamo una `Casella` con una `Move`: a livello di memoria e compilazione, rimangono la medesima struttura.

### 4.3.2 I Tipi di Dato Algebrici (data)

Per forgiare un tipo robusto e totalmente nuovo, interviene la parola chiave `data`. Attraverso essa, definiamo il tipo e i suoi **Costruttori di Dati**.

```
1 -- Costruttori costanti (Tipi Enumerati)
2 data Bool = False | True
3 data SetteNani = Eolo | Pisolo | Brontolo | Cucciolo
4 data Giorni = Lun | Mar | Mer | Gio | Ven | Sab | Dom
5
6 -- Tipi Parametrici (Polimorfi)
7 data Product a b = P a b
8 data Either a b = Left a | Right b
```

A differenza del sinonimo, la `data` fornisce isolamento totale: tentare di passare `Eolo` a una funzione che aspetta `Bool` genera un rigoroso errore a tempo di compilazione.

I costruttori di dati diventano a tutti gli effetti il fondamento per il pattern matching:

```
1 altezza Cucciolo = 99
2 altezza _       = 124
3
4 myFst (P x y) = x
```

## 4.4 Definizione e Derivazione di Istanze

Quando coniamo un nuovo tipo tramite `data`, esso nasce "nudo": non si può confrontare, non si può stampare e non si può ordinare. Per dotarlo di questi poteri, dobbiamo dichiararlo istanza delle varie classi.

```
1 instance Eq Bool where
2     False == False = True
3     True  == True  = True
4     _     == _     = False
```

Scrivere questi blocchi noiosi per ogni tipo è una perdita di tempo. Per le classi fondamentali, Haskell introduce la **derivazione automatica** (`deriving`):

```
1 data Giorni = Lun | Mar | Mer | Gio | Ven | Sab | Dom
2     deriving (Eq, Ord, Show, Enum, Read)
```

La clausola `deriving` fa magie dietro le quinte:

- `Ord` inferisce l'ordinamento in base a come abbiamo scritto i costruttori (`Lun < Mar` sarà `True`).
- `Show` genera una funzione che stampa esattamente il nome testuale del costruttore (`"Lun"`).
- `Enum` fa capire ad Haskell che il tipo è enumerabile in passi discreti, abilitando l'uso delle range list come `lavorativi = [Lun .. Ven]`.

## 4.5 Gestione del Fallimento: Il tipo Maybe

### Concetto Chiave: La liberazione dal NULL

Nei linguaggi imperativi, le operazioni che falliscono restituiscono spesso puntatori nulli (`null`), causando i famigerati crash a runtime. In Haskell, per codificare il concetto di fallimento o assenza di valore, si utilizza in modo type-safe il tipo polimorfo `Maybe`.

```
1 data Maybe a = Just a | Nothing
```

Una computazione corretta viene impacchettata in `Just valore`, mentre un'eccezione logica viene mappata su `Nothing`. L'esempio classico è la divisione protetta:

```
1 safediv _ 0 = Nothing
2 safediv n m = Just (n `div` m)
```

Per usare il risultato di `safediv`, il chiamante è obbligato a livello di compilatore ad aprire il pacco usando il pattern matching, scrivendo codice sia per il ramo `Just` sia per il ramo `Nothing`. Questo paradigma previene le disastrose eccezioni non gestite.

## 4.6 Tipi Induttivi e Strutture Dati Ricorsive

La parola chiave `data` permette l'autodefinizione. Questo genera i Tipi Induttivi (o ricorsivi), la base per creare qualsiasi struttura dinamica:

```
1 -- I Numeri Naturali (Assiomatizzazione di Peano)
2 data Nat = Zero | Succ Nat
3
4 -- La definizione pura di Lista Omogenea
5 data List a = Nil | Cons a (List a)
6
7 -- Un Albero Binario (F = Foglia, R = Radice)
8 data BTree a = F a | R a (BTree a) (BTree a)
```

La costruzione di funzioni su queste entità fluisce con una naturalezza disarmante tramite la ricorsione sui costruttori:

```
1 -- Calcolo della lunghezza di una lista custom
2 myLength Nil = Zero
3 myLength (Cons _ coda) = Succ (myLength coda)
4
5 -- Visita In-Order (Flatten) di un albero binario
6 flatten (F x) = Cons x Nil
7 flatten (R val left right) =
8     append (flatten left) (Cons val (flatten right))
```



## 4.7 Oltre il Binario: Alberi Ennari ed Espressioni

Le strutture dati si combinano perfettamente con i tipi standard di Haskell, comprese le liste, per formare architetture complesse. Ad esempio, un **albero n-ario** (dove ogni nodo radice possiede una lista arbitraria di sotto-alberi) si definisce con straordinaria eleganza in un'unica riga:

```
1 data Tree a = T a [Tree a]
```

E raccogliere tutte le etichette di questo albero infinito? Basta usare la funzione `concat` e `map` per delegare la ricorsione ai sotto-alberi in cascata:

```
1 leafs (T x []) = [x]    -- Nodo senza sotto-alberi = foglia
2 leafs (T x sottoalberi) = concat (map leafs sottoalberi)
```

Un ultimo miracolo di espressività è la possibilità di stoccare **funzioni** all'interno dei nodi. Utilizzando il tipo `Either` (che sceglie deterministicamente o un tipo A o un tipo B), possiamo definire un albero che modella un'espressione matematica: un nodo radice contiene la funzione operatore, mentre le foglie contengono i valori numerici.

```
1 -- O e' una funzione (Num -> Num -> Num), oppure e' un Num
2 type Exp a = BTree (Either (a -> a -> a) a)
3
4 expEval :: Num a => Exp a -> a
5 -- Valutazione: estrae f e la applica ricorsivamente a sinistra e
   destra
6 expEval (R (Left f) left right) = f (expEval left) (expEval right)
7 expEval (F (Right v)) = v
```

Questa compenetrazione fluida tra dati e funzioni, unita al pattern matching distruttivo, fa di Haskell un linguaggio impareggiabile per lo sviluppo di interpreti e processori formali.

# L'onere (e l'onore) della Prova: Dimostrazioni

---

## 5.1 L'Algebra dei Programmi Funzionali

In matematica siamo abituati a manipolare espressioni algebriche per semplificarle o per dimostrare equivalenze. Consideriamo il classico prodotto notevole:

$$(a + b)(a - b) = a(a - b) + b(a - b) = a^2 - ab + ba - b^2 = a^2 - b^2$$

Queste trasformazioni non sono meri esercizi di stile: hanno un profondo **impatto computazionale**. L'espressione originale richiede un'addizione, una sottrazione e una moltiplicazione; l'espressione finale richiede due elevamenti a potenza e una sottrazione. A livello hardware, una forma può essere calcolata molto più velocemente dell'altra.

Nella programmazione funzionale pura, grazie alla totale assenza di side-effects (trasparenza referenziale), i programmi *sono* espressioni matematiche. Possiamo applicare un **ragionamento equazionale** al codice sorgente: sostituendo le chiamate a funzione con le loro definizioni, possiamo dimostrare rigorosamente la correttezza degli algoritmi e, parallelamente, derivarne versioni computazionalmente più efficienti.

## 5.2 Induzione Strutturale: L'Esponenziale sui Naturali

Per dimostrare proprietà su tipi induttivi, si ricorre al **Principio di Induzione**. Utilizziamo i Numeri Naturali (definiti come `Zero` o `Succ Nat`) per rendere la struttura induttiva cristallina. Vogliamo dimostrare che l'esponenziale trasforma le somme in prodotti:  $m^{p+q} = m^p \cdot m^q$ . In Haskell, questo teorema si traduce nell'equazione:

$$\text{exp } m \text{ (add } p \text{ } q) = \text{mul } (\text{exp } m \text{ } p) (\text{exp } m \text{ } q)$$

Procediamo per induzione sulla variabile  $q$ :

### 1. Caso Base ( $q = \text{Zero}$ ):

- *Parte sinistra:* `exp m (add p Zero)`. Per la definizione di `add`, sommare `Zero` a  $p$  restituisce  $p$ . L'espressione diventa `exp m p`.
- *Parte destra:* `mul (exp m p) (exp m Zero)`. Per definizione, `exp m Zero` fa 1 (`Succ Zero`). Moltiplicare un numero per 1 tramite `mul` restituisce il numero stesso. L'espressione si riduce a `exp m p`.

Le due parti coincidono.

2. **Passo Induttivo** ( $q = \text{Succ } n$ ): Assumiamo per Ipotesi Induttiva (I.I.) che la proprietà valga per  $n$ .

- *Parte sinistra*:  $\text{exp } m (\text{add } p (\text{Succ } n))$ . Per definizione di **add**, l'espressione diventa  $\text{exp } m (\text{Succ } (\text{add } p n))$ . Per definizione di **exp**, portiamo fuori un  $m$ :  $\text{mul } m (\text{exp } m (\text{add } p n))$ .
- *Parte destra*:  $\text{mul } (\text{exp } m p) (\text{exp } m (\text{Succ } n))$ . Sviluppando l'esponentiale più a destra, diventa  $\text{mul } (\text{exp } m p) (\text{mul } m (\text{exp } m n))$ . Sfruttando la proprietà commutativa di **mul**, possiamo estrarre  $m$ :  $\text{mul } m (\text{mul } (\text{exp } m p) (\text{exp } m n))$ .

Osservando i blocchi finali a sinistra e a destra, notiamo che l'interno delle parentesi rappresenta esattamente il nostro teorema istanziato su  $n$ , che è strutturalmente più piccolo di  $\text{Succ } n$ . Applicando l'ipotesi induttiva, l'identità è dimostrata (CVD).

### Risvolti Computazionali: La Moltiplicazione Egiziana

Queste prove simboliche ci permettono di implementare versioni ultra-rapide degli algoritmi. Se sappiamo che  $m^{p+q} = m^p \cdot m^q$ , possiamo dedurre che se  $n$  è pari (ovvero  $n = q + q$ ), allora  $m^n = m^q \cdot m^q$ . Questo ci permette di dimezzare il numero di moltiplicazioni a ogni step, calcolando l'esponentiale in tempo logaritmico  $\Theta(\log n)$ . Questo algoritmo è storicamente noto come *Moltiplicazione Egiziana*.

```

1 exp m 0 = 1
2 exp m 1 = m
3 exp m n
4   | r == 0      = es * es
5   | otherwise = m * es * es
6   where
7       (q, r) = n 'divMod' 2
8       es = exp m q

```

## 5.3 L'Induzione sulle Liste e il Lemma di reverse

Anche per le liste finite si utilizza l'induzione. Il *Caso Base* è la lista vuota  $[]$ , mentre il *Passo Induttivo* si basa sul pattern  $(x:xs)$ .

Proviamo a dimostrare che **reverse** è un'involuzione, ovvero che **reverse** (**reverse**  $xs$ ) =  $xs$ . Nel passo induttivo ( $xs = x : xs'$ ), sostituendo otteniamo:

$$\text{reverse } (\text{reverse } (x : xs')) = \text{reverse } (\text{reverse } xs' ++ [x])$$

A questo punto ci blocciamo: non possiamo spingere **reverse** dentro la parentesi perché non conosciamo le regole di interazione tra **reverse** e **++**. Abbiamo bisogno di un **Lemma** preliminare.

Dimostriamo per induzione sul primo argomento che **reverse** ( $xs ++ ys$ ) = **reverse**  $ys ++$  **reverse**  $xs$ .

- **Caso Base** ( $xs = []$ ): **reverse** ( $[] ++ ys$ ) = **reverse**  $ys$ . A destra: **reverse**  $ys ++$  **reverse**  $xs$  = **reverse**  $ys ++ []$  = **reverse**  $ys$ . Le due parti equivalgono.
- **Passo Induttivo** ( $xs = z : zs$ ): **reverse** ( $(z : zs) ++ ys$ ) = **reverse** ( $z : (zs ++ ys)$ ). Per la clausola di **reverse**, portiamo  $z$  in fondo: **reverse** ( $zs ++ ys$ ) ++  $[z]$ .

Applicando l'I.I., l'interno diventa `reverse ys ++ reverse zs`. Aggiungendo `[z]` in coda tramite associatività, ricaviamo esattamente `reverse ys ++ reverse (z : zs)`.

Ora che abbiamo il lemma, possiamo completare agilmente la dimostrazione dell'involution originale.

## 5.4 Funtori, Strictness e Trasformazioni Naturali

Una funzione si definisce **strict** (stretta) se forza la valutazione del suo argomento. In termini algebrici,  $f$  è stretta se e solo se `f undefined = undefined`. Se sappiamo che una funzione  $f$  è stretta, allora possiamo estrarla in modo sicuro dalle funzioni decostruttive come `head`:

$$\text{head } (\text{map } f \text{ } xs) = f (\text{head } xs)$$

La funzione `map` gode di un'altra eccezionale proprietà: è un *funtore*. Conserva l'identità (`map id = id`) e la composizione. Dimostriamo che `map (f ∘ g) = map f ∘ map g` nel caso base `[]`:

- A sinistra: `map (f ∘ g) [] = []`.
- A destra: `(map f ∘ map g) [] = map f (map g []) = map f [] = []`.

Queste proprietà si estendono a una famiglia di equazioni chiamate **trasformazioni naturali** (provenienti dalla Teoria delle Categorie), che sono vere a prescindere dal tipo specifico dei dati contenuti nelle liste:

```
1 map f . tail    = tail . map f          -- se f e' stretta
2 map f . reverse = reverse . map f
3 map f . concat  = concat . map (map f)
4 map concat      = concat . concat
```

## 5.5 Anatomia dei Riduttori: foldr vs foldl

I funzionali di riduzione astragano gli schemi di ricorsione sulle liste.

- `foldr` (fold right) sostituisce i costruttori in loco e raccoglie i risultati partendo dal caso base a destra:

$$\text{foldr } (\oplus) \text{ } v \text{ } [x, y, z] = x \oplus (y \oplus (z \oplus v))$$

- `foldl` (fold left) spinge i calcoli *in avanti* durante la discesa, fungendo da iteratore con accumulatore:

$$\text{foldl } (\oplus) \text{ } v \text{ } [x, y, z] = ((v \oplus x) \oplus y) \oplus z$$

### Proprietà di foldr: La Distributività

Un teorema potentissimo afferma che `foldr` distribuisce un operatore  $\oplus$  sull'operatore di concatenazione `++`:

$$\text{foldr } f \ e \ (xs \ ++ \ ys) = \text{foldr } f \ e \ xs \ \oplus \ \text{foldr } f \ e \ ys$$

Affinché questa equazione sia sempre valida, è stato dimostrato per induzione che devono sussistere due condizioni stringenti:

1.  $e$  deve essere l'elemento neutro (sinistro) dell'operatore  $\oplus$ .
2. Deve valere che  $f \ x \ (y \oplus z) = (f \ x \ y) \oplus z$  (che è sempre vero se  $f = \oplus$  ed  $\oplus$  è associativo).

Da questo teorema deriva istantaneamente che  $\text{sum } (xs \ ++ \ ys) = \text{sum } xs + \text{sum } ys$ , semplicemente applicando il fatto che `sum` è definita come `foldr (+) 0`.

## 5.6 Complessità Spaziale, Temporale e Lazy Evaluation

Sebbene per operatori associativi con elemento neutro (come `+`) valga l'identità `foldl (+) e = foldr (+) e`, le due funzioni esibiscono comportamenti di efficienza drammaticamente diversi.

**1. Valutazione Pigra ed Espansione in Memoria** Valutando `foldl (+) 0 [1, 2, 3]`, Haskell (essendo lazy) non esegue subito le somme matematiche, ma alloca nella memoria heap un immenso "thunk" non valutato:

$$\text{foldl } (+) \ (0+1) \ [2, 3] \rightarrow \text{foldl } (+) \ ((0+1)+2) \ [3] \rightarrow (((0+1)+2)+3)$$

La computazione matematica reale comincia *solo* quando la ricorsione termina, comportando un pesante overhead di memoria. Al contrario, `foldr` restituisce subito il primo costruttore valutabile.

**2. Efficienza Asintotica: L'esempio dell'Append** Siano  $x, y, w, z$  delle liste di lunghezza  $m, n, p, q$ . Vediamo cosa succede se usiamo i riduttori per concatenarle:

- `foldl (++) [] [x, y, w, z] = ((([] ++ x) ++ y) ++ w) ++ z`. Dato che il costo di `++` dipende solo dal primo argomento, gli elementi di  $x$  verranno percorsi 3 volte, quelli di  $y$  2 volte, ecc. Il costo totale è  $\Theta(3m + 2n + p)$ .
- `foldr (++) [] [x, y, w, z] = x ++ (y ++ (w ++ (z ++ [])))`. In questo caso, ogni elemento delle liste viene scorso *esattamente una sola volta*. Il costo crolla a  $\Theta(m + n + p)$ .

**3. Ottimizzazione di reverse** Possiamo applicare questa conoscenza per ottimizzare l'involuzione per eccellenza: il ribaltamento di una lista. Implementare `reverse` come ricorsione ingenua in coda richiede tempo quadratico  $\Theta(n^2)$ . Se però usiamo `foldl` come un iteratore imperativo che scorre la lista da sinistra verso destra, "spingendo" ogni elemento in testa al nuovo accumulatore tramite il `cons` rovesciato (`flip (:)`), otteniamo un'efficienza lineare:

```
1 reverseLinear xs = foldl (flip (:)) [] xs
```

# Ideologia Funzionale in Azione: Comporre, Comporre, Comporre

---

## 6.1 Sistemi di Voto: Un approccio Top-Down

Il cuore della programmazione funzionale è la **composizione**. Quando affrontiamo un problema reale, l'obiettivo non è scrivere un singolo blocco monolitico di codice che calcola il risultato, ma immaginare una "catena di montaggio" (pipeline) in cui i dati fluiscono attraverso una serie di trasformazioni via via più raffinate.

Consideriamo un semplice **Voting System**: abbiamo una lista di voti (stringhe con i nomi dei candidati) e vogliamo determinare il vincitore. Invece di usare cicli `for` e contatori, il Vero Programmatore Haskell ragiona per macro-operazioni:

1. Generare una lista di coppie (`numeroVoti`, `candidato`).
2. Ordinare questa lista. Poiché l'ordine standard sulle tuple in Haskell è lessicografico (guarda prima il primo elemento, poi il secondo), mettendo il numero di voti come primo elemento della tupla, la lista verrà ordinata per preferenze crescenti.
3. Estrarre l'ultimo elemento della lista (il candidato con più voti).
4. Restituire solo il nome del candidato estraendo il secondo elemento della tupla.

Tutta questa logica si condensa in una singola riga di eleganza assoluta:

```
1 winner :: Ord a => [a] -> a
2 winner = snd . last . sort . results
```

Il lavoro "sporco" viene delegato alla funzione `results`, che possiamo implementare accoppiando una lista di candidati unici con i rispettivi conteggi:

```
1 results :: Eq a => [a] -> [(Int, a)]
2 results xs = zip (map (\x -> count x xs) cdts) cdts
3   where
4       count x = length . filter (==x)
5       cdts = rmvDups xs
6
7 rmvDups [] = []
8 rmvDups (x:xs) = x : rmvDups (filter (/=x) xs)
```

Questo stile *Top-Down* favorisce un "rapid-prototyping": componiamo funzioni standard come `zip`, `map`, `filter` e `length` senza preoccuparci inizialmente dell'efficienza, per ottenere rapidamente codice corretto e dimostrabile.

### 6.1.1 Sistemi Elettorali Complessi e Iterazione

Immaginiamo un sistema elettorale più complesso, in cui ogni elettore fornisce non un singolo voto, ma una *lista di preferenze* in ordine. Il vincitore si ottiene eliminando iterativamente i candidati che ricevono meno "prime scelte", finché non ne rimane uno solo.

Nei linguaggi imperativi useremmo un ciclo `while`. In Haskell, possiamo mimare l'iterazione usando funzionali predefiniti come `until`. Questo funzionale accetta un predicato di stop, una funzione di trasformazione dello stato e uno stato iniziale: `until p f x` calcola ripetutamente  $f(x)$ ,  $f(f(x))$ , ... finché il risultato non soddisfa il predicato  $p$ .

```
1 until :: (a -> Bool) -> (a -> a) -> a -> a
2 until p f a = if p a then a else until p f (f a)
```

L'intero algoritmo si riduce all'applicazione di `until`, definendo una funzione che elimina il peggior candidato dal turno corrente (`voteRound`) e un predicato che verifica se è rimasto un solo candidato:

```
1 winnerB'' = fst . until noOne voteRound
```

## 6.2 Propaganda e Verità: Il caso del Quicksort

I manuali di programmazione funzionale esibiscono spesso l'algoritmo di ordinamento *Quicksort* come il trofeo supremo dell'espressività del linguaggio. Sfruttando la *list comprehension*, il codice si scrive letteralmente in 4 righe:

```
1 qSort [] = []
2 qSort (x:xs) = qSort smaller ++ [x] ++ qSort larger
3   where
4     smaller = [a | a <- xs, a <= x]
5     larger  = [b | b <- xs, b > x]
```

### Concetto Chiave: Quicksort Funzionale vs Imperativo

Sebbene questo codice sia immensamente più leggibile rispetto all'equivalente in C o Java, nasconde delle insidie prestazionali. La forza teorica del Quicksort tradizionale risiede nell'essere *in-place*: non alloca memoria extra, scambiando gli elementi direttamente tramite puntatori in  $\Theta(1)$ . Nel codice Haskell appena visto, stiamo allocando in continuazione nuove liste in memoria per `smaller` e `larger`, e stiamo usando l'operatore `++` che ha un costo lineare sulla prima lista. Questa implementazione funzionale rinuncia all'efficienza spaziale in nome della purezza matematica e dell'immutabilità dei dati.

## 6.3 Elaborazione Testi: Il problema delle Common Words

Un esercizio classico (che fece la storia dell'Informatica in un celebre scontro intellettuale tra Donald Knuth e Douglas McIlroy) richiede di estrarre le  $n$  parole più frequenti da un lungo testo testuale, stampandole in ordine di frequenza.

In Haskell, applichiamo nuovamente l'ideologia della "pipeline di trasformazioni", progettando mentalmente il flusso dei tipi di dato prima ancora di scrivere il codice:

$$\text{Text} \xrightarrow{\text{findWords}} [\text{Word}] \xrightarrow{\text{countFreqs}} [(\text{Int}, \text{Word})] \xrightarrow{\text{sort} \dots} \text{String}$$

1. **Trovare e normalizzare le parole:** Non ci limitiamo a usare la `words` di `Prelude`, perché mantiene la punteggiatura. Creiamo una tabella di traduzione (`zip ['a'..'z'] ['A'..'Z']`) e scriviamo una funzione ricorsiva `toLowerAlpha` che filtra e converte in minuscolo in una singola passata.
2. **Contare le frequenze:** Inizialmente potremmo usare la nostra funzione `results` scritta in precedenza, che ha complessità  $\Theta(n^2)$ . Se però il testo è enorme (es. la Divina Commedia), l'algoritmo esplode.
3. **Ottimizzazione (Raffinamento):** Possiamo passare da  $\Theta(n^2)$  a  $\Theta(n \log n)$ . L'idea geniale è di ordinare alfabeticamente la lista delle parole *prima* di contarle. Così facendo, le parole identiche si troveranno contigue. A questo punto, basta usare la funzione di libreria `span` (che divide una lista in due parti basandosi su un predicato) per scorrere la lista linearmente una volta sola:

```
1 countSeqs [] = []
2 countSeqs (x:xs) = (1 + length uguali, x) : countSeqs diversi
3   where
4     (uguali, diversi) = span (==x) xs
```

Questa è la quintessenza dell'Ingegneria del Software funzionale: comporre funzioni modulari, dimostrarne la correttezza matematica, e infine isolare i colli di bottiglia sostituendoli con equivalenti ottimizzati, senza toccare la logica architetturale del programma.

## 6.4 Problemi Combinatori: Esplorare lo Spazio di Ricerca

Haskell eccelle nella risoluzione di problemi combinatori, dove è necessario generare l'intero spazio delle soluzioni. I costrutti *lazy* permettono di definire l'insieme di tutte le soluzioni possibili e poi estrarre solo quelle di interesse.

### 6.4.1 L'Insieme delle Parti (Powerset)

Calcolare tutti i sottoinsiemi di una lista richiede un ragionamento strettamente induttivo. I sottoinsiemi di una lista `x:xs` si dividono in due categorie:

- Quelli che *non* contengono `x` (che sono banalmente i sottoinsiemi di `xs`).
- Quelli che *contengono* `x` (che si ottengono prendendo tutti i sottoinsiemi di `xs` e aggiungendovi `x` in testa).

```
1 powerset [] = [[]]
2 powerset (x:xs) = map (x:) ts ++ ts
3   where ts = powerset xs
```



## 6.4.2 Combinazioni di k elementi

Seguendo lo stesso principio induttivo (noto in matematica come la ricorrenza del Triangolo di Tartaglia per il coefficiente binomiale), le combinazioni di lunghezza  $k$  per  $x:xs$  sono l'unione tra: le combinazioni di lunghezza  $k - 1$  prese dalla coda  $xs$  (a cui anteponiamo  $x$ ), e le combinazioni di lunghezza  $k$  prese direttamente dalla coda  $xs$  (che escludono  $x$ ).

## 6.4.3 Anagrammi e Permutazioni

Come generiamo tutte le permutazioni di una stringa (es. "roma")? Il ragionamento è: per ogni elemento  $x$  della lista, metto  $x$  in prima posizione e lo faccio seguire da tutte le permutazioni della lista rimanente (la lista originale a cui ho rimosso  $x$ ). Matematicamente, se indichiamo con  $xs_{-i}$  la lista privata dell' $i$ -esimo elemento, le permutazioni sono:  $\bigcup_{i=1}^n x_i \bullet \text{perm}(xs_{-i})$ .

Traduciamo questo formalismo direttamente in codice:

```
1 delete _ [] = []
2 delete x (y:ys)
3   | x == y    = ys -- Rimuove solo la prima occorrenza
4   | otherwise = y : delete x ys
5
6 permutations [] = [[]]
7 permutations xs = concat (map (\x -> map (x:) (permutations (delete x
   xs))) xs)
```

L'unione insiemistica ( $\bigcup$ ) diventa `concat`, mentre il mettere in testa all'elemento estratto ( $\bullet$ ) si traduce nel funzionale annidato `map (\x -> map (x:) ...)`.

### Alternative Eleganti: xsMeno1

Invece di usare `delete`, un approccio ancora più compatto consiste nel generare ricorsivamente la lista di tutte le possibili estrazioni (una lista di liste dove in ognuna manca l' $i$ -esimo elemento). Questo approccio si combina perfettamente con `zipWith`, rimuovendo la necessità della lambda esplicita per l'accoppiamento e portando a un codice di una bellezza formale quasi assoluta.

# In principio era il Lambda Calcolo

## 7.1 Anima e Corpo della Computazione

Nel 1931, il logico Alonzo Church introdusse il  $\lambda$ -calcolo nel tentativo di formalizzare il concetto matematico di calcolabilità effettiva. In matematica classica, una funzione  $f : A \rightarrow B$  è concepita puramente a livello semantico (la sua "anima"): è definita estensionalmente come un sottoinsieme del prodotto cartesiano  $A \times B$ .

Tuttavia, quando scriviamo  $f(x) = x^2 + 1$ , stiamo descrivendo una *procedura meccanica di calcolo*. Il  $\lambda$ -calcolo cattura questo aspetto sintattico e operativo (il "corpo"). La sua **Sintassi** è definita dalla seguente grammatica BNF, che genera i cosiddetti  $\lambda$ -termini  $(M, N)$ :

$$M, N ::= x \mid \lambda x.M \mid (M \ N)$$

1. **Variabile**  $(x)$ : Rappresenta un parametro o un valore.
2. **Astrazione**  $(\lambda x.M)$ : Crea una funzione anonima che lega la variabile  $x$  nel corpo  $M$ .
3. **Applicazione**  $((M \ N))$ : Rappresenta la chiamata della funzione  $M$  sull'argomento  $N$ .

Per alleggerire le formule, si adottano convenzioni sintattiche rigorose:

- **L'Applicazione associa a sinistra**:  $M \ N \ P \equiv ((M \ N) \ P)$ .
- **L'Astrazione associa a destra**:  $\lambda xyz.M \equiv \lambda x.(\lambda y.(\lambda z.M))$ . Questa proprietà permette di rappresentare funzioni a più argomenti valutandole un argomento alla volta (*Curryficazione*).

## 7.2 Variabili, $\alpha$ -conversione e Sostituzione formale

L'ambiente di un termine è determinato dalle sue variabili. L'insieme delle **Variabili Libere**  $FV(M)$  è definito induttivamente:

- $FV(x) = \{x\}$
- $FV(\lambda x.M) = FV(M) \setminus \{x\}$
- $FV(M \ N) = FV(M) \cup FV(N)$

### La $\alpha$ -conversione e la Sostituzione

Il nome di una variabile legata è influente:  $\lambda x.x$  è identico a  $\lambda y.y$ . Questa equivalenza si chiama  **$\alpha$ -conversione**. Si assume sempre la *convenzione di Barendregt*: i nomi delle variabili legate vengono scelti tutti diversi tra loro e diversi da quelli delle variabili libere.

L'operazione fondamentale della computazione è la **sostituzione**  $M[N/x]$  (leggasi: " $M$  in cui  $x$  è sostituito da  $N$ "). Si definisce rigorosamente così:

1.  $x[N/x] = N$
2.  $y[N/x] = y$  se  $y \neq x$
3.  $(M P)[N/x] = (M[N/x] P[N/x])$
4.  $(\lambda y.M)[N/x] = \lambda y.(M[N/x])$  se  $y \neq x$  e  $y \notin FV(N)$

L'ultima condizione evita la *cattura delle variabili*: se  $y$  fosse libera in  $N$ , sostituendo  $N$  dentro l'astrazione  $\lambda y$  finirebbe per essere legata erroneamente.

Il motore del calcolo è la  **$\beta$ -riduzione**. Un termine della forma  $(\lambda x.M)N$  è un  $\beta$ -redex. La computazione è l'applicazione della regola:

$$(\lambda x.M)N \rightarrow_{\beta} M[N/x]$$

## 7.3 Combinatori e Strutture Dati Logiche

I termini chiusi ( $FV(M) = \emptyset$ ) sono detti **combinatori**. In  $\lambda$ -calcolo i dati primitivi non esistono: logica e strutture si codificano come puro flusso di controllo.

### 7.3.1 Booleani e Condizionali

Definiamo i valori di verità come funzioni di scelta:

- **Vero** /  $K$ :  $T \equiv \lambda xy.x$  (Cancellatore: restituisce il primo e ignora il secondo).
- **Falso** /  $O$ :  $F \equiv \lambda xy.y$  (Restituisce il secondo e ignora il primo).

L'istruzione **if x then M else N** è l'applicazione  $x M N$ . Operatori logici:

- **NOT**:  $\lambda x.x F T$ .
- **AND**:  $\lambda xy.x y F$ .

### 7.3.2 Tuple e Proiettori

Una tupla  $(M_1, M_2, \dots, M_k)$  si codifica incapsulando gli elementi in un'astrazione in attesa di un selettore:

$$\lambda x.x M_1 M_2 \dots M_k$$

Per estrarre il  $j$ -esimo elemento, si costruisce un **Proiettore**  $\pi_j^k \equiv \lambda x_1 \dots x_k.x_j$ . Se passiamo il proiettore alla tupla, otteniamo:

$$(\lambda x.x M_1 \dots M_k) \pi_j^k \rightarrow \pi_j^k M_1 \dots M_k \rightarrow M_j$$

## 7.4 Aritmetica di Church

I numeri naturali vengono codificati come **iteratori**. Il numerale di Church  $\underline{n}$  applica una funzione  $s$  per  $n$  volte a una base  $z$ :

$$\underline{n} \equiv \lambda s z. s(s(\dots(s\ z)\dots))$$

- $\underline{0} \equiv \lambda s z. z$  (Equivale al combinatore  $F$ )
- $\underline{1} \equiv \lambda s z. s\ z$

Le operazioni manipolano le iterazioni:

- **Successore:**  $succ \equiv \lambda x s z. s\ (x\ s\ z)$ .
- **Addizione:**  $add \equiv \lambda m n s z. m\ s\ (n\ s\ z)$ .
- **Moltiplicazione:**  $mul \equiv \lambda m n s. m\ (n\ s)$ .

## 7.5 Ricorsione Primitiva e i Suoi Limiti

Possiamo definire funzioni iterando i numerali di Church. Un concetto fondamentale è la **Ricorsione Primitiva**. Una funzione  $h$  è definita per ricorsione primitiva da  $f$  e  $g$  se:

$$h(n+1) = f\ n\ (h\ n) \quad \text{e} \quad h(0) = g$$

Il calcolo del fattoriale o del predecessore ricade in questa categoria. Tuttavia, le funzioni primitive ricorsive equivalgono ai programmi scrivibili solo con cicli **for** predeterminati. Non coprono l'intera calcolabilità.

Esistono funzioni calcolabili che *non* sono esprimibili per ricorsione primitiva, come la **Funzione di Ackermann** (che cresce a una velocità impensabile):

- $A(0, n) = n + 1$
- $A(m + 1, 0) = A(m, 1)$
- $A(m + 1, n + 1) = A(m, A(m + 1, n))$

Questa funzione è totale (termina sempre, poiché l'ordine lessicografico delle coppie  $(m, n)$  decresce costantemente), ma per implementarla serve la ricorsione generale (il ciclo **while**).

## 7.6 Combinatori di Punto Fisso

Per supportare la ricorsione generale (funzioni anonime che richiamano se stesse), il  $\lambda$ -calcolo usa il **Combinatore di Punto Fisso**  $Y$ , il quale garantisce  $YM \rightarrow M(YM)$ .

- **Curry** ( $Y_C$ ):  $\lambda f. (\lambda x. f(x\ x))(\lambda x. f(x\ x))$ .
- **Turing** ( $Y_T$ ):  $\theta\theta$  dove  $\theta \equiv \lambda xy. y(xy)$ .

Per implementare una funzione ricorsiva come il fattoriale  $f = \text{corpo}(f)$ , in  $\lambda$ -calcolo non si fa altro che astrarre la  $f$ :  $\lambda f. \text{corpo}(f)$  e passarvi un combinatore di punto fisso:  $Y_T\ (\lambda f. \text{corpo}(f))$ , innescando l'infinito nastro della ricorsione pura.

## 7.7 Tipaggio in Haskell: Il Sistema Hindley-Milner

Il  $\lambda$ -calcolo puro non ha tipi, permettendo forme divergenti come  $\Omega \equiv (\lambda x.x\ x)(\lambda x.x\ x)$ . Haskell adotta il sistema di tipi Hindley-Milner, diviso in:

1. **Monotipi** ( $\tau$ ):  $\tau ::= \text{Int} \mid \text{Bool} \mid \alpha \mid \tau_1 \rightarrow \tau_2$ .
2. **Schemi di Tipo / Politipi** ( $\sigma$ ):  $\sigma ::= \tau \mid \forall \alpha. \sigma$ .

### Sostituzione e Unificazione di Tipi

Una **sostituzione**  $\theta$  è una funzione finita che associa tipi a variabili di tipo (es.  $\theta = \{(\alpha, \text{Int})\}$ ). Applicare  $\theta$  a  $\tau$  ( $\tau\theta$ ) significa rimpiazzare le variabili in  $\tau$ . Due tipi **unificano** se esiste una sostituzione  $\theta$  tale che  $\tau_1\theta = \tau_2\theta$ . Inoltre, diciamo che un tipo  $\tau_1$  è **più generale** di  $\tau_2$  ( $\tau_1 \leq \tau_2$ ) se esiste una sostituzione  $\theta$  tale per cui  $\tau_1\theta = \tau_2$ . L'interprete GHCi basa l'inferenza esattamente su questo algoritmo di unificazione.

### 7.7.1 Regole di Derivazione

Il giudizio  $\Gamma \vdash M : \sigma$  indica che sotto l'ambiente  $\Gamma$ , il termine  $M$  ha tipo  $\sigma$ . Le regole principali:

- **(VAR)**:

$$\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma}$$

- **(ABS)**:

$$\frac{\Gamma \cup \{x : \tau_1\} \vdash M : \tau_2}{\Gamma \vdash \lambda x.M : \tau_1 \rightarrow \tau_2}$$

- **(APP)**:

$$\frac{\Gamma \vdash M : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash N : \tau_1}{\Gamma \vdash M\ N : \tau_2}$$

- **(FIX)**:

$$\frac{\Gamma \cup \{x : \tau\} \vdash M : \tau}{\Gamma \vdash \text{fix } x.M : \tau}$$

(Questa regola serve a tipare le definizioni ricorsive native del linguaggio senza passare per il combinatore  $Y$ ).

Operazioni sui quantificatori:

- **(GEN)**:

$$\frac{\Gamma \vdash M : \sigma \quad \alpha \notin \text{free}(\Gamma)}{\Gamma \vdash M : \forall \alpha. \sigma}$$

(Generalizzazione del tipo).

- **(SPEC)**:

$$\frac{\Gamma \vdash M : \sigma \quad \sigma \leq \sigma'}{\Gamma \vdash M : \sigma'}$$

(Specializzazione del tipo polimorfo).

### 7.7.2 L'Asimmetria del LET e l'Occurs Check

Nel  $\lambda$ -calcolo, `let  $x = N$  in  $M$`  equivale a  $(\lambda x.M)N$ . In Haskell, le loro regole di tipo **non coincidono**.

La regola per l'astrazione (ABS) unita all'applicazione (APP) esige che i parametri siano **monotipi**. Se valutiamo  $(\lambda x.x\ x)(\lambda y.y)$ , la prima  $x$  attende  $\tau_1 \rightarrow \tau_2$  e la seconda è passata come argomento di tipo  $\tau_1$ . Ne consegue  $\tau_1 = \tau_1 \rightarrow \tau_2$ . Questa equazione circolare causa il temibile errore di **occurs check** (tipi infiniti) e la compilazione fallisce.

Al contrario, la regola del LET accetta **politipi**:

$$\frac{\Gamma \vdash N : \sigma_1 \quad \Gamma \cup \{x : \sigma_1\} \vdash M : \sigma_2}{\Gamma \vdash \text{let } x = N \text{ in } M : \sigma_2} \text{ (LET)}$$

Valutando `let  $x = \lambda y.y$  in  $x\ x$` , il compilatore generalizza  $N$  in  $\forall \alpha. \alpha \rightarrow \alpha$ . Quando scende nel corpo, sa che  $x$  è uno schema di tipo. Può dunque specializzare (SPEC) separatamente le due occorrenze di  $x$ : la prima in  $(\beta \rightarrow \beta) \rightarrow (\beta \rightarrow \beta)$  e la seconda in  $\beta \rightarrow \beta$ . Il codice compila, sbloccando il polimorfismo let-bound che conferisce ad Haskell la sua ineguagliabile potenza.

# L'Importanza di Essere Lazy: Efficienza ed Espressività

---

## 8.1 La Fusion Law per foldr

Come visto nei capitoli precedenti, il ragionamento equazionale permette di dimostrare proprietà formali sui programmi. Esiste una forma di induzione generalizzata e preconfezionata, pensata appositamente per il funzionale `foldr`, nota come **Fusion Law** (Legge di Fusione). Lo scopo della Fusion Law è ottimizzare il codice fondendo due operazioni applicate in sequenza (di cui almeno una è un `foldr`) in un'unica passata sulla lista.

Esempi classici di applicazione di questa legge sono:

- `double . sum = foldr ((+) . double) 0`
- `length . concat = foldr ((+) . length) 0`

L'equazione generale del teorema che vogliamo dimostrare ha questa forma:

$$f \circ \text{foldr } g \ a = \text{foldr } h \ b$$

### Come leggere la dimostrazione (e cos'è una funzione stretta)

Prima di addentrarci nei calcoli, è fondamentale capire *come* si affronta una dimostrazione di questo tipo in Haskell e fissare alcuni concetti base.

**1. Il Ragionamento Equazionale:** Per ricavare le condizioni (i vincoli) che rendono vera l'equazione qui sopra, si procede per analisi dei casi, valutando i tre possibili "stati" in cui può trovarsi una lista in Haskell:

- La lista vuota (`[]`).
- Una lista con almeno un elemento (`x:xs`).
- Una lista divergente o "rotta" (`undefined`).

Per ciascuno di questi tre casi, prenderemo la parte sinistra dell'equazione (Sinistra) e la svilupperemo applicando passo-passo le definizioni delle funzioni coinvolte. Poi faremo lo stesso con la parte destra (Destra). Infine, confronteremo i due risultati ottenuti: le condizioni necessarie affinché la Sinistra sia identica alla Destra diventeranno i tre vincoli del nostro teorema.

**2. Le Funzioni Strette (Strict):** Nel terzo caso dell'induzione incontreremo il valore `undefined`, che rappresenta una computazione che va in loop infinito o genera un'eccezione (un crash). Grazie alla valutazione pigra (lazy) di Haskell, *non è detto* che passare

`undefined` a una funzione faccia crashare il programma. Ad esempio, una funzione costante `k x = 5` ignorerà totalmente il parametro `x`: se chiamiamo `k undefined`, il risultato sarà tranquillamente `5`. Una **funzione stretta** (strict) è invece una funzione che *ha assolutamente bisogno* di valutare il suo parametro per poter procedere. Se le passiamo un parametro "rotto", lei si rompe a sua volta. Matematicamente, si definisce stretta una funzione  $f$  tale per cui:

$$f \text{ undefined} = \text{undefined}$$

## La Dimostrazione Passo-Passo

Ora che abbiamo gli strumenti, applichiamo il ragionamento ai tre casi per ricavare i vincoli della Fusion Law:

### 1. Caso `[]` (Lista vuota):

- *Sinistra:* Valutiamo  $f (\text{foldr } g \ a \ [])$ . Per definizione, `foldr` su lista vuota restituisce il caso base  $a$ . Otteniamo quindi  $f \ a$ .
- *Destra:* Valutiamo  $\text{foldr } h \ b \ []$ . Per definizione, otteniamo  $b$ .

Uguagliando le due parti, otteniamo il primo vincolo:  $\mathbf{f \ a = b}$ .

### 2. Caso `(x:xs)` (Passo induttivo):

- *Sinistra:* Valutiamo  $f (\text{foldr } g \ a \ (x : xs))$ . Per la definizione di `foldr` induttivo, l'espressione si espande in  $f (g \ x (\text{foldr } g \ a \ xs))$ .
- *Destra:* Valutiamo  $\text{foldr } h \ b \ (x : xs)$ . Si espande in  $h \ x (\text{foldr } h \ b \ xs)$ .

Ora usiamo l'ipotesi induttiva: assumiamo che il teorema sia già valido per la coda `xs`. Quindi a destra possiamo sostituire  $\text{foldr } h \ b \ xs$  con  $f (\text{foldr } g \ a \ xs)$ . La parte destra diventa:  $h \ x (f (\text{foldr } g \ a \ xs))$ . Se confrontiamo Sinistra e Destra, notiamo che l'unica differenza è l'operatore più esterno. Sostituendo il blocco ricorsivo ( $\text{foldr } g \ a \ xs$ ) con una variabile generica  $y$ , otteniamo il secondo vincolo:  $\mathbf{f \ (g \ x \ y) = h \ x \ (f \ y)}$ .

### 3. Caso `undefined` (Liste parziali):

- *Sinistra:* Valutiamo  $f (\text{foldr } g \ a \ \text{undefined})$ . Poiché `foldr` deve smontare la lista per fare pattern matching, la sua esecuzione su `undefined` crasha, restituendo `undefined`. L'espressione diventa  $f \ \text{undefined}$ .
- *Destra:* Valutiamo  $\text{foldr } h \ b \ \text{undefined}$ . Anche in questo caso il risultato è `undefined`.

Uguagliando, ricaviamo che deve valere  $f \ \text{undefined} = \text{undefined}$ . Ovvero, il terzo vincolo richiede che  $f$  **sia una funzione stretta**.



### Teorema: Fusion Law per foldr

Siano  $f, g, h$  funzioni e  $a, b$  valori. Se valgono le seguenti tre condizioni:

1.  $f$  è stretta ( $f \text{ undefined} = \text{undefined}$ )
2.  $f a = b$
3.  $f (g x y) = h x (f y)$

Allora vale l'identità:  $f \circ \text{foldr } g a = \text{foldr } h b$

## 8.2 Induzione su Liste Parziali

Una **lista parziale** è una lista che non termina con il costruttore `[]` ma con una computazione divergente (`undefined`). Il principio di induzione su liste parziali richiede di dimostrare:

- Caso base:  $P(\text{undefined})$  è vera.
- Passo induttivo: Se  $P(xs)$  è vera (con  $xs$  parziale), allora  $P(x : xs)$  è vera.

Dimostriamo che  $xs ++ ys = xs$  se  $xs$  è una lista parziale:

- **Caso Base:**  $\text{undefined} ++ ys = \text{undefined}$  (perché  $++$  ha bisogno di ispezionare il primo argomento, quindi crasha subito).
- **Passo Induttivo:**  $(x : xs) ++ ys = x : (xs ++ ys)$ . Applicando l'ipotesi induttiva su  $xs$  parziale (che sappiamo fare  $xs$ ), l'espressione diventa  $x : xs$ . CVD.

**Attenzione:** molte proprietà valide per liste finite falliscono miseramente su liste parziali. Prendiamo l'involuzione `reverse` (`reverse xs`) =  $xs$ . Nel caso base, valutare `reverse undefined` dà `undefined`. L'equazione diventa `undefined = undefined`, e qui regge. Tuttavia, nel passo induttivo avremo: `reverse (x:xs) = reverse xs ++ [x]`. Per ipotesi induttiva, sappiamo che `reverse xs = undefined`, quindi ci ritroviamo a calcolare `undefined ++ [x]`. Poiché l'operatore `++` fallisce se il primo argomento è `undefined`, l'intero risultato crolla a `undefined`. L'elemento  $x$  viene letteralmente inghiottito dall'infinito! La proprietà di involuzione non vale sulle liste parziali.

## 8.3 Dentro la Lazy Evaluation: Valori e Forme Normali

Per capire a fondo la laziness, dobbiamo tornare al  $\lambda$ -calcolo e definire cosa sia un "valore", ovvero un termine che non necessita di ulteriori riduzioni. Esistono diversi livelli di valutazione:

1. **Normal Form (nf):** Un termine senza alcun  $\beta$ -redex (totalmente valutato). Es.  $I = \lambda x.x$ .
2. **Head Normal Form (hnf):** Un termine chiuso nella forma  $\lambda x_1 x_2 \dots x_n. x_i N_1 N_2 \dots N_m$ , dove i sotto-termini  $N$  non sono necessariamente in forma normale.
3. **Weak Head Normal Form (whnf):** Una forma normale "debole" di testa. Estrae solo la dipendenza funzionale principale (una lambda all'esterno) oppure un costruttore esplicito.

Il termine  $\lambda y.\Omega$  è in whnf (anche se il corpo andrebbe in loop infinito, non lo si esplora).

### 8.3.1 I Valori in Haskell

Essendo basato sulla strategia *call-by-name*, Haskell elegge a "valori" le espressioni in **Weak Head Normal Form (WHNF)**. In pratica:

- Una funzione è un valore. Non si valuta il suo corpo interno finché non le sono stati forniti gli argomenti necessari.
- **Un costruttore di dati è un valore in WHNF**, indipendentemente dal fatto che i suoi campi siano valutati!

Sono perfettamente in WHNF espressioni come:

- `(3+2) : merge [1,2,3] [3,4]` (Il costruttore `:` è il valore esterno, il resto è in sospenso).
- `undefined : undefined`
- `Just (3+2)`

La funzione chiamante determina quanto "a fondo" valutare. `print` esige la forma normale per stampare l'intero risultato. `head` esige solo la WHNF: le basta vedere che c'è un costruttore `:` per estrarne il primo elemento. Per questo motivo, l'istruzione `length (take 2 [undefined, undefined, undefined])` restituisce tranquillamente 2, senza mai generare errori! La `take` si limita a spaccettare il costruttore lista due volte senza toccare i valori interni.

## 8.4 Il Problema delle Duplicazioni e lo Sharing

### Lazyness

Gli ingredienti per la lazyness sono 3: *call-by-name*, *weak head normal form* e *sharing*.

La strategia *Call-by-Name* ritarda la valutazione, ma rischia di ricalcolare espressioni complesse se una funzione duplica i suoi parametri. Sia `sqr x = x * x`. Valutiamo `sqr (sqr (3+4))`:

**In Call-by-Name (inefficiente):**

1. `sqr (3+4) * sqr (3+4)`
2. `((3 + 4) * (3 + 4)) * sqr (3+4)`
3. `(7 * (3 + 4)) * ... ==> (7 * 7) * ... ==> 49 * sqr (3+4)`
4. Il calcolo di `(3 + 4)` e della `sqr` esterna deve essere calcolato due volte!

**In Call-by-Value (linguaggi imperativi):**

1. `sqr (sqr (7))`
2. `sqr (7 * 7)`
3. `sqr (49) ==> 49 * 49 = 2401`

Per salvare la laziness senza perdere efficienza, Haskell implementa lo **Sharing** (condivisione) tramite Grafi Aciclici Diretti (DAG) anziché semplici alberi sintattici. Sotto il cofano, l'interprete converte i parametri in blocchi `let` invisibili:  $\text{sqr } x \implies \text{let } y = x \text{ in } y * y$ .

#### In Call-by-Name con Sharing (Haskell):

1. `let z = (3 + 4) in (let y = z * z in y * y)`
2. L'interprete valuta `z` una sola volta (7).
3. `let y = 7 * 7 in y * y`. Valuta `y` una sola volta (49).
4. Esegue  $49 * 49 = 2401$ .

#### Regola Aurea dell'Efficienza

Una valutazione lazy con sharing **non esegue mai** più riduzioni dell'equivalente valutazione strict. Il vantaggio straordinario è che può eseguirne infinitamente di meno! Ad esempio, `head (map (*2) [1..1000])` in Haskell esegue un solo prodotto, risparmiando 999 operazioni inutili rispetto a linguaggi come C o Python.

## 8.5 I Teoremi di Confluenza e Standardizzazione

In presenza di computazioni infinite (come  $\Omega$ ), l'ordine di valutazione decide se il programma restituirà un valore o andrà in stallo. Prendiamo l'espressione  $K I \Omega$ . Se usiamo un approccio che forza i parametri, proveremmo a valutare  $\Omega$  entrando in loop per sempre. Ma se usiamo la Call-by-Name (ridurre sempre il redex più esterno a sinistra per primo), valutiamo prima il combinatore  $K$ :

$$K I \Omega \equiv (\lambda xy.x) I \Omega \rightarrow I$$

Il programma termina subito, ignorando del tutto l'infinito!

Questi principi sono sorretti da due capisaldi dell'informatica teorica:

- **Teorema di Church-Rosser (Confluenza):** L'ordine delle riduzioni non altera mai la correttezza del risultato. Se un termine  $M$  può essere ridotto in  $N'$  e in  $N''$ , esisterà sempre un termine unificatore  $P$  a cui sia  $N'$  che  $N''$  arriveranno. Corollario: le forme normali sono uniche.
- **Teorema di Standardizzazione:** Se esiste una qualsiasi sequenza di operazioni in grado di condurre un'espressione a una forma normale (un risultato finale), la strategia Call-by-Name la troverà sempre in un numero finito di passi, garantendo la massima probabilità di terminazione possibile per il programma.

## 8.6 Espressività: Funzioni Non Strette e Operatori Logici

La laziness fornisce ai programmatori Haskell un'arma preclusa ad altri linguaggi: la capacità di definire funzioni utente "non strette" capaci di recuperare ed eludere gli errori dei parametri. L'esempio cardine è la funzione `null`:

```
1 myNull [] = True
2 myNull _ = False
```

Questa banale funzione esibisce comportamenti strabilianti:

- `myNull [undefined] ==> False` (Valuta solo il costruttore esterno `:`, ignora l'infinito all'interno).
- `myNull undefined ==> *** Exception` (Essendo la lista stessa rotta, il pattern matching fallisce).

Tale elasticità permette di ridefinire liberamente l'operatore `||` (OR logico). In C/Java, l'operatore `||` gode del meccanismo di *corto-circuito* fornito direttamente dal compilatore. In Haskell, è un operatore definibile dal programmatore:

```
1 -- LEFT OR (Corto-circuita a sinistra, come in C)
2 leftOr True _ = True
3 leftOr _     x = x
4
5 -- EAGER OR (Costretto a valutare entrambi gli argomenti)
6 eagerOr True  True  = True
7 eagerOr False True  = True
8 eagerOr True  False = True
9 eagerOr False False = False
10
11 -- RIGHT OR (Corto-circuita a destra)
12 rightOr = flip leftOr
```

Comportamento in caso di fallimento:

- `leftOr True undefined ==> True` (Il programma è salvo, il corto-circuito ha funzionato).
- `leftOr undefined True ==> *** Exception` (Il programma crasha dovendo testare la sinistra).
- `eagerOr True undefined ==> *** Exception` (La valutazione esplicita obbliga a calcolare entrambi i parametri, condannando l'esecuzione).

### 8.6.1 Il limite della Sequenzialità: Il Parallel OR

Questa analisi svela un limite architetturale. Il ruolo dei parametri in Haskell non è simmetrico. Da un punto di vista puramente logico esisterebbe il **Parallel OR** (`por`):

```
1 por undefined True = True
2 por True undefined = True
```

In Haskell standard **questo è impossibile da definire**. L'interprete è intrinsecamente deterministico e sequenziale: processa i pattern dall'alto verso il basso e valuta i parametri da sinistra verso destra. Per realizzare `por`, l'interprete dovrebbe lanciare l'esecuzione dei due parametri in thread concorrenti, restituendo il risultato e uccidendo il thread in errore non appena il primo dei due restituisce `True`.

# Funzionali sì, ma efficienti! (Spazio e Tempo)

## 9.1 Il Costo dell'Astrazione

Un celebre e pungente aforisma di Alan Perlis recita: *"I programmatori funzionali conoscono il valore di ogni cosa, ma non il costo di nulla"*. Se da un lato la programmazione funzionale pura ci regala un codice estremamente elegante, matematicamente dimostrabile e privo di effetti collaterali, dall'altro l'abuso di astrazioni e della *lazy evaluation* può nascondere gravissimi problemi di inefficienza. Ignorare come l'interprete gestisce la memoria porta inevitabilmente a programmi che esauriscono lo spazio (Space Leaks) o che impiegano tempo esponenziale per operazioni banali.

In questo capitolo esploreremo le tecniche fondamentali per "scoperchiare il motore" di Haskell, imparando a ottimizzare algoritmi inefficienti e a domare la valutazione pigra.

## 9.2 Tecnica 1: Tupling (Accoppiamento)

Spesso, le definizioni matematiche ricorsive più naturali portano a ricalcoli inutili o a percorsi multipli e ridondanti sulle strutture dati. La tecnica del **Tupling** consiste nel restituire più valori contemporaneamente impacchettandoli in una tupla, permettendo di portare avanti i calcoli in un'unica passata.

### 9.2.1 L'Esempio di Fibonacci

La definizione matematica della sequenza di Fibonacci è l'esempio principe di inefficienza temporale:

```
1 fib 0 = 0
2 fib 1 = 1
3 fib n = fib (n-1) + fib (n-2)
```

Questa implementazione genera un albero di ricorsione esponenziale  $O(2^n)$ , ricalcolando gli stessi valori innumerevoli volte. Per abbattere la complessità a  $O(n)$ , usiamo il Tupling per restituire due valori consecutivi della serie:

```
1 fibT n = fst (fibPair n)
2   where
3     fibPair 0 = (0, 1)
4     fibPair n = let (a, b) = fibPair (n-1)
5                  in (b, a+b)
```

### 9.2.2 Zucchero Sintattico: Gli operatori \$ e \$!

Per rendere il codice più pulito ed evitare foreste di parentesi, Haskell introduce l'operatore di applicazione `$`. La sua definizione e la sua priorità (la più bassa possibile, zero) sono dichiarate così:

```
1 infixr 0 $
2 ($) :: (a -> b) -> a -> b
3 f $ x = f x
```

Scrivere `f (g (h x))` equivale a scrivere elegantemente `f $ g $ h x`.

Esiste anche la variante `$!`, che esegue un'applicazione *eager* (ansiosa/stretta), forzando la valutazione del parametro prima di passarlo alla funzione:

```
1 infixr 0 $!
2 ($!) :: (a -> b) -> a -> b
3 f $! x = x 'seq' f x
```

Possiamo riscrivere l'algoritmo di Fibonacci  $O(n)$  mimando un ciclo `for` imperativo tramite questi operatori:

```
1 -- Iteratore generico
2 for f 0 x = x
3 for f n x = for f (n-1) (f x)
4
5 -- Fibonacci iterativo con tupla
6 fib n = fst $ for g n $ (0,1)
7   where g (a, b) = (b, a+b)
```

## 9.3 Tecnica 2: Accumulating Parameters

Un'altra fonte classica di inefficienza è l'uso ingenuo dell'operatore di concatenazione `++`, che ha un costo lineare proporzionale alla lunghezza del suo primo argomento.

Se implementiamo il rovesciamento di una lista come `reverse (x:xs) = reverse xs ++ [x]`, la complessità esplode a  $O(n^2)$ . Per risolvere, usiamo un **Parametro Accumulatore**: una variabile aggiuntiva passata alla funzione ricorsiva che "costruisce" il risultato passo dopo passo.

```
1 reverse xs = revAcc xs []
2   where
3     revAcc [] acc = acc
4     revAcc (x:xs) acc = revAcc xs (x : acc)
```

Inserire in testa all'accumulatore costa  $O(1)$ . Ripetuto  $n$  volte, otteniamo un `reverse` lineare  $O(n)$ .

La stessa logica si applica al *flatten* (lo "schiacciamento") di un albero binario:

```
1 -- Naif: O(n^2) a causa del ++ annidato
2 flatten (F x) = [x]
3 flatten (R x t1 t2) = flatten t1 ++ (x : flatten t2)
4
5 -- Ottimizzato con Accumulatore: O(n)
6 flattenAcc (F x) acc = x : acc
7 flattenAcc (R x t1 t2) acc = flattenAcc t1 (x : flattenAcc t2 acc)
```

```

8
9 flattenTree t = flattenAcc t []

```

## 9.4 Il Lato Oscuro della Laziness: Space Leaks

Haskell valuta le espressioni solo quando strettamente necessario. Questo ha un prezzo oscuro nella gestione della memoria. Ricordiamo il comportamento di `foldl`:

$$\text{foldl } (+) 0 [1, 2, 3] \implies (((0 + 1) + 2) + 3)$$

Poiché l'addizione non viene forzata finché il risultato finale non viene stampato o utilizzato, l'interprete costruisce nell'Heap un'enorme struttura dati astratta chiamata **thunk** per memorizzare le operazioni in sospenso. Se la lista ha milioni di elementi, questo thunk consuma tutta la RAM, portando a un blocco del sistema noto come **Space Leak** (perdita di spazio).

### 9.4.1 Forzare la Mano: Il costrutto `seq` e `foldl'`

Per sconfiggere lo space leak, dobbiamo obbligare Haskell a valutare il risultato parziale *immediatamente*. Per farlo si usa la primitiva `seq :: a -> b -> b`, che valuta `a` fino alla sua **Weak Head Normal Form (WHNF)** e poi restituisce `b`.

Applicando `seq` all'accumulatore, otteniamo `foldl'`, la versione *strict* di `foldl`:

```

1 foldl' _ a [] = a
2 foldl' f a (x:xs) = let a' = f a x
3                      in a' `seq` foldl' f a' xs

```

A ogni passo, il nuovo accumulatore `a'` (es.  $0 + 1$ ) viene forzato a diventare un numero puro (1) prima di procedere. I thunk non si formano più e il calcolo avviene in memoria costante  $O(1)$ .

#### Equivalenza tra `foldl` e `foldr`

Possiamo calcolare in modo efficiente la lunghezza di una lista senza space leaks usando `foldl'`. Ma quando possiamo sostituire con certezza un `foldr` con un `foldl`? La correttezza matematica stabilisce che `foldl` ( $\oplus$ )  $e = \text{foldr} ( $\otimes$ )  $e$  a patto che valgano le seguenti condizioni sull'elemento neutro  $e$  e sugli operatori:$

1.  $(x \otimes y) \oplus z = x \otimes (y \oplus z)$
2.  $e \oplus x = x \otimes e$

## 9.5 Il Tranello delle Tuple

Vediamo un caso reale in cui l'ottimizzazione sembra perfetta, ma nasconde un'insidia letale. Supponiamo di calcolare la media dei valori di una lista:

```
1 mean [] = 0
2 mean xs = sum xs / fromIntegral (length xs)
```

Questa versione naïf scandisce la lista due volte (una per `sum`, una per `length`). Uniamo Tupling e `foldl'` per fare una singola passata efficiente:

```
1 mean' xs = s / l
2   where (s, l) = foldl' (\(somma, len) x -> (somma+x, len+1)) (0, 0)
               xs
```

Abbiamo usato `foldl'`, quindi siamo al sicuro dallo Space Leak, giusto? **Assolutamente no!**

Qui si manifesta l'inganno più sottile della **Weak Head Normal Form (WHNF)**. Ricordiamo che `seq` (usato dentro `foldl'`) valuta un'espressione solo fino al suo costruttore più esterno. Quando `foldl'` applica `seq` alla nuova tupla `(somma+x, len+1)`, l'interprete valuta fino a incontrare il costruttore della tupla `(,)`. A quel punto si ferma, considerandolo un valore valido. I valori interni `somma+x` e `len+1` **non vengono calcolati!** Si creano milioni di thunk annidati *all'interno* della tupla, che divoreranno inesorabilmente la memoria.

### Come salvare la Tupla (Strict Tupling)

Per risolvere questo disastro, non basta forzare il contenitore (la tupla), ma bisogna forzare esplicitamente il contenuto *prima* di impacchettarlo.

```
1 meanStrict xs = s / l
2   where
3     (s, l) = foldl' passo (0,0) xs
4     passo (somma, len) x =
5       let s' = somma + x
6         l' = len + 1
7       in s' `seq` l' `seq` (s', l')
```

Costringendo l'interprete a valutare prima `s'` e `l'`, eviteremo la formazione di thunk interni, ottenendo l'efficienza  $O(1)$  in spazio e  $O(n)$  in tempo.

## 9.6 Derivazione Formale di Programmi: Il miracolo di `scanl`

L'algebra dei programmi non serve solo a dimostrare proprietà, ma permette di trasformare funzioni corrette ma inefficienti in algoritmi ottimali. La funzione `scanl` calcola i risultati parziali di un `foldl` applicato a tutti i prefissi di una lista. La sua specifica formale naïf è:

$$\text{scanl } f \ e \ xs = \text{map } (\text{foldl } f \ e) \ (\text{inits } xs)$$

Esempio: `scanl (+) 0 [1,2,3] → [0,1,3,6]`.

Poiché `inits` restituisce  $n$  sottoliste e `foldl` le ripercorre tutte, questa specifica costa  $O(n^2)$ . Possiamo derivare una versione  $O(n)$  ragionando per induzione sulla lista `xs`:



### 1. Caso base ([]):

`scanl f e []`

Sostituiamo la definizione: `map (foldl f e) (inits [])`. I prefissi di una lista vuota sono solo la lista contenente la lista vuota, ovvero  `[[] ]`:

`map (foldl f e) [[]]`

Applicando la `map`, otteniamo: `[foldl f e []]`. Per definizione di `foldl` sul caso base, questo fa `e`. Il risultato è `[e]`. **Prima equazione derivata:** `scanl f e [] = [e]`

### 2. Passo induttivo (x:xs):

`scanl f e (x : xs)`

Sostituiamo la definizione: `map (foldl f e) (inits (x : xs))`. Sappiamo che i prefissi di `(x : xs)` sono la lista vuota `[]` seguita da tutti i prefissi di `xs` a cui è stata anteposta `x` (cioè `map (x :) (inits xs)`). Quindi l'espressione diventa:

`map (foldl f e) ([] : map (x :) (inits xs))`

Applichiamo la `map` esterna distribuendola sulla testa e sulla coda:

`(foldl f e []) : map (foldl f e) (map (x :) (inits xs))`

La testa si riduce a `e`. Per la coda, sfruttiamo la proprietà di composizione dei funtori (`map A ∘ map B = map (A ∘ B)`):

`e : map (foldl f e ∘ (x :)) (inits xs)`

Il termine composto `(foldl f e ∘ (x :))` applicato a una generica lista `ys` fa `foldl f e (x : ys)`. Ma per la definizione di `foldl`, calcolare un fold partendo dall'accumulatore `e` su `(x : ys)` equivale a ricalcolare il fold sulla coda `ys` usando il nuovo accumulatore aggiornato `(f e x)`! Quindi possiamo riscrivere la composizione come `foldl f (f e x)`. L'espressione diventa:

`e : map (foldl f (f e x)) (inits xs)`

Se osserviamo bene, la parte a destra del `cons (:)` corrisponde *esattamente* alla specifica originale di `scanl`, ma con il nuovo accumulatore `(f e x)` e la lista accorciata `xs`.

**Seconda equazione derivata:** `scanl f e (x : xs) = e : scanl f (f e x) xs`

Senza dover inventare nulla a livello imperativo, le fredde e rigorose regole della sostituzione matematica ci hanno guidato alla stesura dell'algoritmo ricorsivo lineare ottimale.

# L'Infinita Importanza di Essere Lazy: Strutture Dati Infinite

---

## 10.1 Oltre il Finito: Stream e Definizioni Circolari

Una delle applicazioni più affascinanti ed eleganti della valutazione *lazy* in Haskell è la possibilità di maneggiare strutture dati infinite, chiamate anche **stream**. Nei linguaggi *eager* (a valutazione rigorosa come C o Java), tentare di istanziare un array infinito porterebbe a un esaurimento istantaneo della memoria. In Haskell, invece, i dati vengono generati "on-demand", permettendoci di definire concetti matematici illimitati.

Possiamo definire una lista contenente infiniti numeri 1 tramite una **definizione circolare**:

```
1 ones = 1 : ones
```

Se chiediamo all'interprete di valutare `ones`, inizierà a stampare infiniti 1 (`[1, 1, 1, 1, ...]`) finché non blocchiamo brutalmente l'esecuzione con `Ctrl+C`. Tuttavia, grazie alla laziness, se usiamo funzioni che richiedono solo un prefisso della lista, il programma terminerà regolarmente e in tempo rapidissimo:

```
1 > take 3 ones
2 [1, 1, 1]
3 > ones !! 111
4 1
```

## 10.2 Algebra e Co-Algebra dei Tipi Ricorsivi

Da un punto di vista formale, gli "abitanti" (i valori) di un tipo ricorsivo in Haskell costituiscono la massima algebra chiusa rispetto all'applicazione dei costruttori. Ad esempio, i numeri naturali (assiomi di Peano) sono il minimo insieme chiuso rispetto al costruttore `Z` (Zero) e `S` (Successore): *"I numeri naturali sono 0 e tutti i successori di un numero naturale. Nient'altro è un numero naturale"*.

### Il Naturale Infinito: $\omega$

Se consideriamo la **co-algebra** (la massima algebra chiusa rispetto ai costruttori), scopriamo che essa contiene anche il "naturale infinito", ovvero  $\omega = S(S(S(\dots))) = S^\infty$ . Possiamo definire questo valore e usarlo nelle computazioni!

```
1 data Nat = Z | S Nat deriving (Show, Eq, Ord)
2 omega = S omega
```

Se proviamo a valutare `omega < S (S Z)`, Haskell risponde immediatamente `False`, poiché esplorando il primo costruttore di `omega` (`S`) si accorge già che è maggiore di un numero finito.

Lo stesso principio si applica alle liste. Eliminando il costruttore base (lista vuota `[]`) e lasciando solo il `Cons`, otteniamo una struttura che può rappresentare *soltanto* liste infinite:

```
1 data ListInf a = C a (ListInf a) deriving (Show, Eq, Ord)
```

## 10.3 Generatori e Zucchero Sintattico

In Haskell possiamo usare lo zucchero sintattico `[0..]` per indicare tutti i numeri naturali. Ma possiamo anche scriverci i nostri generatori. Un approccio tramite parametro di stato:

```
1 nats = nextNat 0
2 where nextNat n = n : nextNat (n+1)
```

Un approccio incredibilmente più elegante che sfrutta la `map` su se stessa:

```
1 nats' = 0 : map (+1) nats'
```

Per le potenze di un numero  $n$ , la prima idea potrebbe essere: `powers n = 1 : map (n*) (powers n)`. Tuttavia, c'è un'alternativa di gran lunga superiore, puramente circolare (senza ricorsione di funzione):

```
1 powers' n = ps
2 where ps = 1 : map (n*) ps
```

In questa versione `ps`, l'interprete capisce che l'oggetto è unico in memoria e deve solo aggiornarlo aggiungendo nodi. Lo stesso trucco si usa per definire la funzione predefinita `repeat`:

```
1 myRepeat x = xs where xs = x : xs
```

## 10.4 Insidie e Produttività: filter e List Comprehension

**Attenzione!** Non bisogna mai dare per scontato che le funzioni base sappiano magicamente di trovarsi di fronte a una lista infinita. Se vogliamo trovare tutti i divisori di un numero  $n$  usando una list-comprehension su un dominio infinito:

```
1 factorsNT n = [x | x <- [1..], n `mod` x == 0]
```

Valutando `factorsNT 24`, il programma stamperà `[1, 2, 3, 4, 6, 8, 12, 24]` ma non terminerà mai (o andrà in loop infinito senza stampare la parentesi chiusa), perché continuerà a cercare per sempre nell'infinita sequenza di numeri successivi per vedere se c'è un altro divisore! La soluzione è usare `takeWhile`, che interrompe la valutazione della lista infinita non appena la condizione diventa falsa:

```
1 factors n = [x | x <- takeWhile (<=n) [1..], n `mod` x == 0]
```

Avendo i divisori in modo sicuro, lo stream dei numeri primi diventa banale:

```
1 primes = [p | p <- [2..], factors p == [1,p]]
```

### Il Concetto di Produttività

Un'equazione ricorsiva su stream deve consumare *meno* input di quanto output produce, altrimenti "si mangia la coda" e non definisce nulla. Ad esempio: `incstnts = (head incstnts) : (tail incstnts)` Questa definizione è inconsistente e causa un blocco. Il processo `incstnts` non genera mai un primo elemento, mentre `head` e `tail` stanno in attesa spasmodica di consumarlo. Si dice che la definizione **non è produttiva**.

## 10.5 Complessità Nascoste: Le tre facce di iterate

La funzione predefinita `iterate` prende una funzione  $f$  e un valore  $x$ , generando lo stream  $[x, f(x), f(f(x)), \dots]$ . Esaminiamo tre possibili implementazioni:

```
1 -- Lineare (Non crea lista ciclica)
2 iterate1 f x = x : iterate1 f (f x)
3
4 -- Lineare (Circolare, riusa l'output in modo ottimale)
5 iterate2 f x = xs where xs = x : map f xs
6
7 -- Quadratica (Disastrosa!)
8 iterate3 f x = x : map f (iterate3 f x)
```

Perché `iterate3` (senza il `where`) è di classe  $O(n^2)$ ? Se valutiamo `iterate3 (*2) 1`, per produrre ogni nuovo elemento dobbiamo rimappare la funzione su tutta la lista generata finora. Per produrre il 4° elemento, l'interprete dovrà calcolare: `map (*2) (map (*2) (map (*2) (...)))`. La mole di `map` annidate cresce linearmente a ogni elemento estratto, portando a uno sforzo quadratico. `iterate2`, grazie al `where`, si allinea all'ultimo elemento prodotto in memoria, costando  $O(1)$  a passo.

## 10.6 Fibonacci e il Crivello di Eratostene

Lo stream di Fibonacci si può scrivere dislocando la lista su se stessa:

```
1 fibs = 0 : 1 : zipWith (+) fibs (tail fibs)
```

Esiste un unico stream `fibs` in memoria. Le `tail` scorrono in avanti, e `zipWith` somma gli elementi accoppiati producendo l'elemento successivo appena in tempo. Per garantire formalmente la produttività, è preferibile la mutua ricorsione guardata (che nasconde l'uso del `tail`):

```

1 fibs' = 0 : fibs''
2 fibs'' = 1 : zipWith (+) fibs' fibs''

```

Un altro capolavoro classico è il **Crivello di Eratostene** in Haskell (ideato da David Turner nel 1975):

```

1 primes = filterPrime [2..]
2   where filterPrime (p:xs) =
3         p : filterPrime [x | x <- xs, x `mod` p /= 0]

```

Questo codice filtra l'infinita lista dei naturali eliminando tutti i numeri divisibili per l'ultimo numero primo trovato, generando magicamente l'infinita sequenza dei numeri primi.

## 10.7 Il Problema degli Hamming Numbers

Vogliamo generare lo stream infinito degli **Hamming Numbers**: tutti i numeri composti esclusivamente dai fattori primi 2, 3 e 5. (Es.  $H = \{2^i \cdot 3^j \cdot 5^k \mid i, j, k \geq 0\}$ ). Vogliamo che lo stream sia in ordine strettamente crescente e senza duplicati.

### 10.7.1 Tentativo 1: Il Filtro Inefficiente

Potremmo scrivere un predicato `compositeOf` che verifica se un numero si divide solo per i fattori dati, e poi usarlo come filtro:

```

1 hamming gs = filter (compositeOf gs) [1..]

```

**Questo approccio è inutilizzabile.** Gli Hamming Numbers hanno densità 0 al crescere di  $n$  (ovvero,  $\lim_{n \rightarrow \infty} H_n/n = 0$ ). Per trovare numeri grandi, il filtro sprecherebbe millenni a setacciare numeri irrilevanti.

### 10.7.2 Tentativo 2: Generazione "Artigianale"

Un modo molto migliore è *generare* la lista dai suoi stessi elementi.

```

1 hamming p q = hs
2   where hs = 1 : nextH (map (p*) hs) (map (q*) hs)
3
4 nextH (hp:hps) (hq:hqs)
5   | hp < hq   = hp : nextH hps (hq:hqs)
6   | hp > hq   = hq : nextH (hp:hps) hqs
7   | otherwise = hp : nextH hps hqs

```

Sfruttando le `map`, abbiamo in memoria due stream derivati da `hs`: uno moltiplicato per `p` e uno per `q`. La funzione `nextH` li ispeziona: sceglie il minimo tra le teste per mantenere l'ordine, e in caso di parità consuma entrambe le teste per rimuovere i duplicati.

### 10.7.3 Tentativo 3: La Soluzione Definitiva

Il concetto di `nextH` non è altro che un **merge ordinato senza duplicati**, che chiameremo `union`:

```

1 union xs@(x:txs) ys@(y:tys)
2   | x < y      = x : union txs ys
3   | x > y      = y : union xs tys
4   | otherwise  = x : union txs tys

```

In matematica, l'insieme di Hamming generalizzato con un set di generatori  $G$  è:  $H = \{1\} \cup \bigcup_{g \in G} \{g \cdot n \mid n \in H\}$ . Traduciamo questa formula letteralmente in Haskell. Per generare tutti gli stream derivati per un set di generatori `gs`, mappiamo la creazione dello stream su `gs`. Dopodiché, fondiamo assieme questa lista di stream usando una riduzione (`foldr1`) basata sulla nostra `union`:

```

1 hamming gs = hs
2   where hs = 1 : foldr1 union (map (\g -> map (g*) hs) gs)

```

**Nota finale:** È cruciale usare `foldr1` (che collassa gli stream partendo da destra in modo lazy) e *non* `foldl`, poiché `foldl` forzerebbe in avanti un'infinita serie di valutazioni prima di poter restituire il primo elemento, non terminando mai su liste infinite!

# Teoria e Pratica degli Stream: Limiti e Generatori Complessi

---

## 11.1 Teoria: Le Liste Infinite come Limiti

Finora abbiamo trattato le liste infinite in modo puramente intuitivo e operativo. Tuttavia, per poter dimostrare formalmente delle proprietà su di esse (ad esempio, dimostrare che due stream generati in modo diverso sono uguali), abbiamo bisogno di un solido supporto matematico. Nella teoria della calcolabilità e della semantica denotazionale, una lista infinita viene definita rigorosamente come il **limite** della sequenza delle sue approssimazioni finite.

Per formalizzare questo concetto, introduciamo il simbolo  $\perp$  (bottom), che in Haskell corrisponde a `undefined`. Questo simbolo denota l'assenza totale di informazione o una computazione divergente. Sui tipi di dato possiamo definire una relazione di **ordinamento dell'informazione** (indicata col simbolo  $\sqsubseteq$ ): diciamo che  $xs \sqsubseteq ys$  se  $xs$  è un'approssimazione (cioè contiene meno informazione o informazione uguale) di  $ys$ .

Ad esempio, consideriamo lo stream dei numeri naturali  $[1..]$ . Esso è il limite della seguente catena di approssimazioni via via crescenti in termini di informazione:

- $\perp$  (Non sappiamo assolutamente nulla della lista)
- $1 : \perp$  (Sappiamo che il primo elemento è 1, ma il resto è ignoto)
- $1 : 2 : \perp$  (Sappiamo che i primi due elementi sono 1 e 2)
- $1 : 2 : 3 : \perp$
- $\dots$
- $1 : 2 : 3 : \dots$  (Il limite infinito, indicato con  $\sqcup_n$ )

### Concetto Chiave: Coerenza e Convergenza

Affinché una sequenza di approssimazioni converga a un limite ben definito (che sia uno stream infinito o una lista finita), la parte "definita" della lista deve stabilizzarsi. Consideriamo la seguente sequenza:

$$\perp, \quad 1 : \perp, \quad 2 : 1 : \perp, \quad 3 : 2 : 1 : \perp, \quad \dots$$

Questa sequenza **non converge a nulla** di sensato. L'informazione fornita è costantemente contraddittoria: il primo elemento cambia a ogni passo (prima è 1, poi è 2, poi è 3). Non c'è alcun limite definibile perché la struttura non si fissa mai. Le liste in Haskell sono costruite come la massima algebra chiusa rispetto ai costruttori, garantendo che i limiti siano ben formati.

## 11.2 La funzione approx e l'Uguaglianza tra Stream

Dato che non possiamo usare l'operatore `==` per confrontare due liste infinite elemento per elemento (il calcolo non terminerebbe mai), come facciamo a dimostrare matematicamente che due stream  $xs$  e  $ys$  sono identici? La teoria dei domini ci fornisce un teorema potentissimo: due stream sono uguali se e solo se *tutte le loro approssimazioni finite sono uguali*.

Per portare questo concetto all'interno di Haskell, definiamo una funzione **approx**, che è un parente stretto della classica **take**, ma con una differenza vitale nella gestione dello zero:

```
1 approx :: Int -> [a] -> [a]
2 approx 0 xs      = undefined  -- Il limite della conoscenza e' bottom
3 approx n []      = []
4 approx n (x:xs) = x : approx (n-1) xs
```

Se la confrontiamo con **take**, notiamo che `take 0 xs = []`. Invece, **approx** restituisce **undefined** ( $\perp$ ) al passo zero, perché rappresenta il taglio dell'informazione. La proprietà fondamentale che governa questa funzione è che una lista coincide con l'unione di tutte le sue approssimazioni:

$$\sqcup_n \text{approx } n \text{ } xs = xs$$

Da questo deriva il **Principio di Co-induzione sulle approssimazioni**:

$$\forall n. \text{approx } n \text{ } xs = \text{approx } n \text{ } ys \implies xs = ys$$

Questo ci autorizza a usare la rassicurante induzione sui numeri naturali  $n$  per dimostrare l'uguaglianza tra entità infinite.

## 11.3 Dimostrazione per Induzione con approx

Mettiamo alla prova il principio dimostrando una proprietà classica. Vogliamo dimostrare che mappare una funzione  $f$  su uno stream generato da **iterate** equivale a generare lo stream partendo da  $f(x)$ :

$$\text{map } f \text{ (iterate } f \text{ } x) = \text{iterate } f \text{ (} f \text{ } x)$$



Dimostriamo per induzione su  $n$  che  $\text{approx } n (\text{map } f (\text{iterate } f x)) = \text{approx } n (\text{iterate } f (f x))$ .

**1. Caso Base ( $n = 0$ ):**

- Sinistra:  $\text{approx } 0 (\text{map } f (\text{iterate } f x)) = \perp$  (per definizione della prima clausola di  $\text{approx}$ ).
- Destra:  $\text{approx } 0 (\text{iterate } f (f x)) = \perp$ .

Le due parti sono identiche (entrambe non definite). Il caso base regge.

**2. Passo Induttivo ( $n + 1$ ):** Assumiamo per Ipotesi Induttiva (I.I.) che la proprietà valga per  $n$ . Sviluppiamo la parte sinistra:

$$\text{approx } (n + 1) (\text{map } f (\text{iterate } f x))$$

Sostituiamo la definizione di  $\text{iterate}$ :

$$= \text{approx } (n + 1) (\text{map } f (x : \text{iterate } f (f x)))$$

Applichiamo la definizione di  $\text{map}$ :

$$= \text{approx } (n + 1) (f x : \text{map } f (\text{iterate } f (f x)))$$

Applichiamo la terza clausola di  $\text{approx}$  (che estrae la testa ed esegue  $\text{approx } n$  sulla coda):

$$= f x : \text{approx } n (\text{map } f (\text{iterate } f (f x)))$$

Ora ci troviamo davanti alla struttura esatta dell'Ipotesi Induttiva! Sostituiamo il blocco con la parte destra dell'I.I.:

$$= f x : \text{approx } n (\text{iterate } f (f (f x)))$$

Mettiamo da parte questo risultato e sviluppiamo ora la parte destra dell'equazione originale del passo  $n + 1$ :

$$\text{approx } (n + 1) (\text{iterate } f (f x))$$

Applichiamo la definizione di  $\text{iterate}$ :

$$= \text{approx } (n + 1) (f x : \text{iterate } f (f (f x)))$$

Applichiamo la terza clausola di  $\text{approx}$ :

$$= f x : \text{approx } n (\text{iterate } f (f (f x)))$$

Il risultato coincide perfettamente con quanto ottenuto dallo sviluppo della parte sinistra. La proprietà è rigorosamente dimostrata per ogni  $n$ , ergo i due stream sono identici.

## 11.4 Pratica: Generatori Complessi e i Numeri di Ulam

Spingiamoci oltre e affrontiamo la creazione di un generatore infinito che richiede l'ispezione della storia passata: i **Numeri di Ulam**. La sequenza di Ulam è definita dalle seguenti regole:

1. I primi due numeri sono  $U_1 = 1$  e  $U_2 = 2$ .
2. Ogni numero successivo  $U_n$  è il più piccolo intero che può essere scritto come somma di due elementi *distinti* precedenti della sequenza, **in un unico modo**.

Analizziamo i primi passi:

- L'unico candidato dopo 1 e 2 è  $1 + 2 = 3$ . C'è un solo modo per farlo, quindi **3 è di Ulam**. (Lista: 1, 2, 3)
- Proviamo a fare somme:  $1 + 3 = 4$  (unico modo). **4 è di Ulam**. (Lista: 1, 2, 3, 4). Nota:  $2 + 2 = 4$  non è ammesso perché gli addendi devono essere distinti.
- Valutiamo le prossime somme:
  - Combinazioni col 1:  $1 + 4 = 5$ .
  - Combinazioni col 2:  $2 + 3 = 5$ ,  $2 + 4 = 6$ .
  - Combinazioni col 3:  $3 + 4 = 7$ .
- Il numero 5 può essere ottenuto sia come  $1 + 4$  che come  $2 + 3$ . Essendoci *due modi*, **5 non è un numero di Ulam**.
- Il numero 6 si ottiene solo come  $2 + 4$  ( $3 + 3$  non vale,  $1 + 5$  non vale perché 5 non è di Ulam). Quindi **6 è di Ulam**.

### 11.4.1 Gestire le Frequenze: I Multi-insiemi

Per implementare l'algoritmo, il nostro generatore deve calcolare tutte le possibili somme future e tenere traccia di *quante volte* ciascuna somma si presenta. Ci serve la struttura del **Multi-insieme** (Multiset), che in Haskell possiamo rappresentare agilmente come una lista ordinata di tuple (valore, frequenza).

Iniziamo definendo due funzioni di supporto. La prima trasforma una lista normale in un multi-insieme in cui ogni elemento ha frequenza 1:

```
1 toMS :: [Int] -> [(Int, Int)]
2 toMS xs = [(x, 1) | x <- xs]
```

La seconda funzione, `unionMS`, è un *merge ordinato* di due multi-insiemi. Se un valore compare in entrambi, le frequenze vengono sommate:

```
1 unionMS :: [(Int, Int)] -> [(Int, Int)] -> [(Int, Int)]
2 unionMS [] ys = ys
3 unionMS xs [] = xs
4 unionMS xs@((x, fx):txs) ys@((y, fy):tys)
5   | x < y      = (x, fx) : unionMS txs ys
6   | x > y      = (y, fy) : unionMS xs tys
7   | otherwise = (x, fx + fy) : unionMS txs tys
```

### 11.4.2 Costruire l'Algoritmo: nextUlam

Per generare lo stream, dobbiamo mantenere uno stato che evolve nel tempo. Questo stato sarà composto da:

- **cs**: La lista dei "candidati", ovvero il multi-insieme di tutte le somme calcolate finora che non sono ancora state elette a numeri di Ulam.
- **i**: Un contatore che indica quanti numeri di Ulam abbiamo trovato finora (serve per sapere quanti elementi prendere per le nuove somme).

L'algoritmo per trovare il prossimo numero di Ulam, dato questo stato e l'accesso all'intero stream dei numeri di Ulam (**us**), si articola così:

1. Esploriamo la lista **cs** dall'inizio (è ordinata per valore crescente) e cerchiamo il primo elemento che possiede frequenza esattamente uguale a 1. Quello sarà il nostro nuovo numero di Ulam, chiamiamolo *u*.
2. Ripuliamo **cs**: rimuoviamo dalla lista dei candidati tutti i valori minori o uguali a *u*, poiché non ci serviranno mai più (gli Ulam numbers sono strettamente crescenti).
3. Calcoliamo le nuove somme prodotte dall'arrivo di *u*. Prendiamo tutti i numeri di Ulam trovati *prima* di *u* (estraendoli in modo safe dallo stream **us** con **takeWhile** (*<u*)) e vi sommiamo *u*. Trasformiamo queste nuove somme in un multi-insieme usando **toMS**.
4. Uniamo il vecchio multi-insieme ripulito con il nuovo multi-insieme appena creato, usando **unionMS**.
5. Effettuiamo la chiamata ricorsiva circolare passando il nuovo stato aggiornato.

Ecco il codice formale e completo:

```
1 ulams :: [Int]
2 ulams = 1 : 2 : nextUlam (toMS [3], 1) ulams
3
4 nextUlam :: ([Int], Int) -> [Int] -> [Int]
5 nextUlam (cs, i) us = u : nextUlam (cs', i+1) us
6   where
7     -- 1. Cerchiamo il minimo candidato con frequenza esattamente 1
8     (u, freq) = head (filter (\(val, f) -> f == 1) cs)
9
10    -- 2. Scartiamo i candidati vecchi o invalidati (minori o uguali a
11    u)
12    cs_pulita = dropWhile (\(val, f) -> val <= u) cs
13
14    -- 3. Generiamo le nuove somme usando in modo lazy i precedenti
15    Ulam
16    newSums = toMS (map (+u) (takeWhile (<u) us))
17
18    -- In alternativa, avendo a disposizione l'indice i, potremmo
19    usare:
20    -- newSums = toMS (map (+u) (take (i+1) us))
21
22    -- 4. Uniamo le vecchie somme (ripulite) con le nuove
23    cs' = unionMS cs_pulita newSums
```

### Analisi della Produttività e Circolarità

Questo codice rappresenta l'apoteosi del paradigma funzionale lazy. Notate la definizione circolare di base: passiamo `ulams` (lo stream intero e parzialmente indefinito) come parametro a `nextUlam`, che lo raccoglie come variabile `us`. Quando all'interno di `nextUlam` andiamo a estrarre i numeri per fare le nuove somme con l'istruzione `takeWhile (<u) us`, non andiamo mai a richiedere elementi indefiniti che manderebbero il programma in stallo! Infatti, per cercare i numeri minori di  $u$ , l'interprete pescherà dallo stream esattamente i numeri che noi stessi abbiamo appena finito di calcolare nei passi precedenti della ricorsione. Il processo consuma sempre un elemento in meno rispetto a quelli disponibili, rendendo l'algoritmo perfettamente **produttivo** e capace di scorrere all'infinito senza blocchi.